

Image and Video Generations

Lecture 4: Classifier-Free Guidance / Latent Diffusion / ControlNet / LoRA

劉育綸

Yu-Lun (Alex) Liu



Guidance Using Additional Information

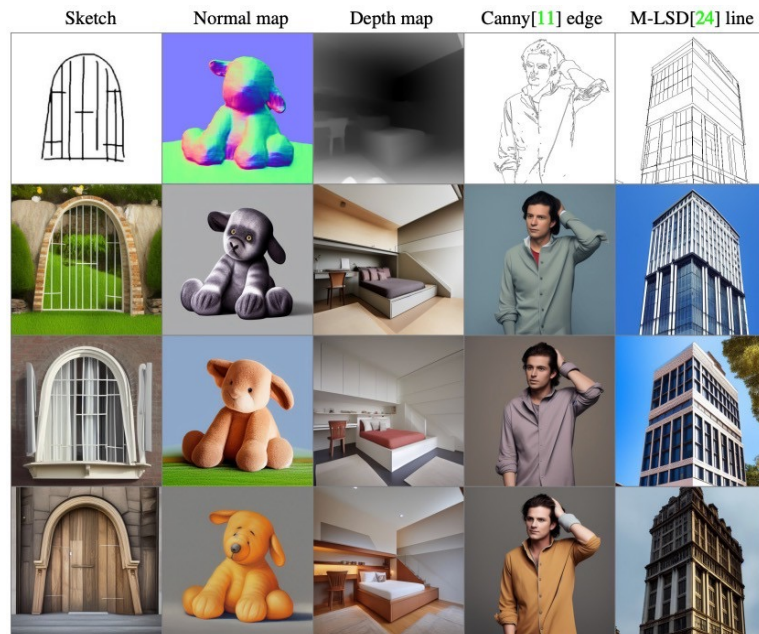
If we have **additional information** about the data, such as class labels for images, can we utilize this information to **improve the quality of generated outputs**?



Ho and Salimans, Classifier-Free Diffusion Guidance, NeurIPS 2021 Workshop.

Fine-Tuning-Based Methods

Can we fine-tune a pretrained diffusion model for conditional generation or personalization?



Zhang et al., Adding Conditional Control to Text-to-Image Diffusion Models, ICCV 2022.



Ruiz et al., DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation, CVPR 2023.

Content

1. Classifier Guidance / Classifier-Free Guidance

2. Fine-tuning-Based Methods

1. ControlNet

2. LoRA

Classifier Guidance /
Classifier-Free Guidance (CFG)

Guidance Using Additional Information

How to use **class labels** or some **additional information** about the data to **improve the quality of generated outputs**?

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



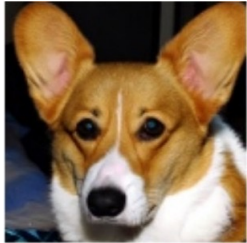
Classifier Guidance

Dhariwal and Nichol, Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021.

Classifier Guidance

Assume that data $\mathbf{x}$ and **class label** y pairs are given:

$$p(\mathbf{x}_t, y) = p(\mathbf{x}_t)p(y|\mathbf{x}_t)$$

(, dog)

Recall: Score Function

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0) = - \frac{\boldsymbol{\varepsilon}_t}{\sqrt{1 - \bar{\alpha}_t}}$$

For the rest of slides, let $p(\mathbf{x}_t) := q(\mathbf{x}_t | \mathbf{x}_0)$:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) = - \frac{\boldsymbol{\varepsilon}_t}{\sqrt{1 - \bar{\alpha}_t}}$$

Classifier Guidance

At each timestep t , we are interested in the **score** function $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t, y)$:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t, y) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$$

$$= -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) + \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$$

$$= -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \left(\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t) \right)$$

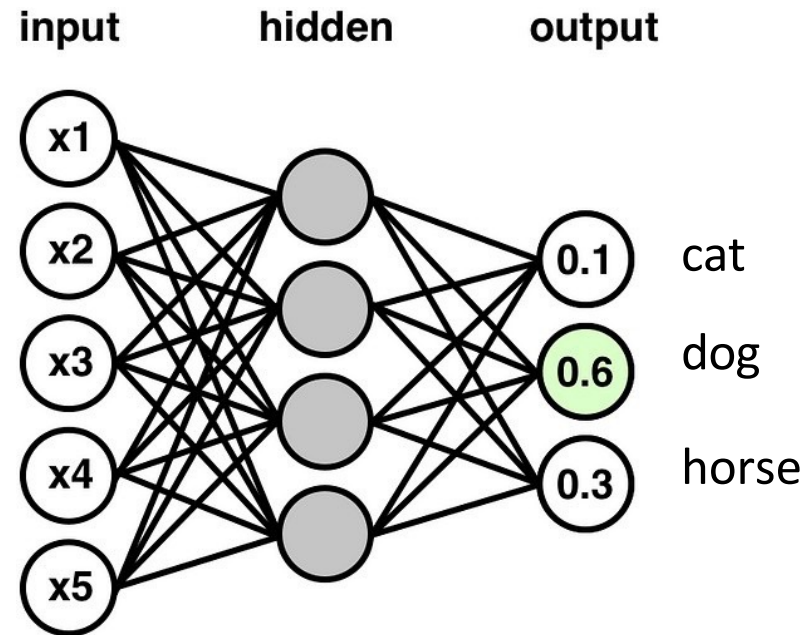
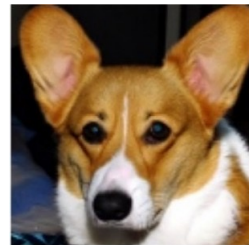
New noise!

How to
compute this?

Classifier Guidance

How to compute $\nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$?

How to learn $p(y|\mathbf{x}_t)$? Train a **classification** network!



Classifier Guidance

How to compute $\nabla_{\mathbf{x}_t} \log p(y|\mathbf{x}_t)$?

- Train a **classifier** $p_\phi(y|\mathbf{x}_t)$, taking noisy data $\mathbf{x}_t$ (and time step t) as input and classifying it.
- Update the noise prediction $\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)$ as follows:

$$\bar{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t) = \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) - \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_\phi(y|\mathbf{x}_t)$$

Condition Enhancement

The **strength** of the classifier guidance can be controlled by adding **weight** $\omega \geq 1$:

$$\bar{\boldsymbol{\varepsilon}}_{\theta}(\boldsymbol{x}_t, t) = \boldsymbol{\varepsilon}_{\theta}(\boldsymbol{x}_t, t) - \omega \sqrt{1 - \bar{\alpha}_t} \nabla_{\boldsymbol{x}_t} \log p_{\phi}(y|\boldsymbol{x}_t)$$

Classifier Guidance

$$\begin{aligned}\bar{\boldsymbol{\varepsilon}}_{\theta}(\mathbf{x}_t, t) &= \boldsymbol{\varepsilon}_{\theta}(\mathbf{x}_t, t) - \omega \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_{\phi}(y|\mathbf{x}_t) \\ &= -\sqrt{1 - \bar{\alpha}_t} (\nabla_{\mathbf{x}_t} \log p_{\phi}(\mathbf{x}_t) + \omega \nabla_{\mathbf{x}_t} \log p_{\phi}(y|\mathbf{x}_t)) \\ &= -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_{\phi}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p_{\phi}(y|\mathbf{x}_t) \omega \\ &= -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log(p_{\phi}(\mathbf{x}_t) p_{\phi}(y|\mathbf{x}_t) \omega)\end{aligned}$$

Classifier Guidance

$$\boxed{p_{\phi}(\mathbf{x}_t)p_{\phi}(y|\mathbf{x}_t)^{\omega}} \text{ instead of } p(\mathbf{x}_t)p(y|\mathbf{x}_t) = p(\mathbf{x}_t|y)$$

$p_{\phi}(y|\mathbf{x}_t)^{\omega}$ amplifies large values.

⇒ Focusing more on the **modes** of the classifier.

⇒ **Higher fidelity** to the input label but **less diversity**.

Classifier Guidance

- (+) Not only is the generation quality improved, but **conditional generation** with labels also becomes possible!
- (+) 可以在任何已訓練好的 unconditional diffusion model 的 inference 時使用，不需要在訓練時就考慮
- (-) Additional training of a classifier is required，且 classifier 的好壞會影響生成效果
- (-) Inference 時需要計算 classifier 的 gradient (較慢)

Classifier-Free Guidance (CFG)

Ho and Salimans, Classifier-Free Diffusion Guidance, NeurIPS 2021 Workshop.

Classifier-Free Guidance (CFG)

How to make a **conditional** diffusion model?

Dog →



Classifier-Free Guidance (CFG)

Modify the noise prediction network to take the condition (label) y as an additional input:

$$\hat{\epsilon}_{\theta}(\mathbf{x}_t, t) \rightarrow \hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}, t)$$

Classifier-Free Guidance (CFG)

We can jointly train the **conditional** and **unconditional** diffusion models by introducing a null label ϕ .

$\hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$: Noise prediction *without* condition.

Classifier-Free Guidance (CFG)

How can we **enhance** the conditioning?

We can **extrapolate** the conditional noise $\hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}, t)$ from the null-condition noise $\hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$:

$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}, t) = (1 + \omega)\hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}, t) - \omega\hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$$

where $\omega \geq 0$.

Connection to Classifier Guidance

Why is it an **extrapolation**?

Let $\lambda := 1 + \omega$ ($\lambda \geq 1$)

$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, y, t) = \lambda \hat{\epsilon}_{\theta}(\mathbf{x}_t, y, t) - (1 - \lambda) \hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$$

$$= \hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t) + \lambda (\hat{\epsilon}_{\theta}(\mathbf{x}_t, y, t) - \hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t))$$

原本 DDPM/DDIM 的
unconditional noise prediction

Conditional noise 相對於
unconditional noise 要增強的部分

Connection to Classifier Guidance

$$\begin{aligned} & \hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t) + \lambda \left(\hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}, t) - \hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t) \right) \\ &= -\sqrt{1 - \bar{\alpha}_t} \left(\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \lambda \left(\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t | \mathbf{y}) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \right) \right) \\ &= -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log \left(p(\mathbf{x}_t) \left(\frac{p(\mathbf{x}_t | \mathbf{y})}{p(\mathbf{x}_t)} \right)^{\lambda} \right) \end{aligned}$$

Connection to Classifier Guidance

$$p(\mathbf{x}_t) \left(\frac{p(\mathbf{x}_t | \mathbf{y})}{p(\mathbf{x}_t)} \right)^\lambda$$

$$\propto p(\mathbf{x}_t) \left(\frac{p(\mathbf{x}_t | \mathbf{y}) \cdot p(\mathbf{y})}{p(\mathbf{x}_t)} \right)^\lambda$$

因為過程中，condition y 是固定的
Multiply constant $p(y)^\lambda$

$$= \boxed{p(\mathbf{x}_t) (p(\mathbf{y} | \mathbf{x}_t))^\lambda}$$

Bayes' theorem

跟 Classifier Guidance 的機率分布一樣

Classifier Guidance

$$p_{\phi}(\mathbf{x}_t)p_{\phi}(y|\mathbf{x}_t)^{\omega} \text{ instead of } p(\mathbf{x}_t)p(y|\mathbf{x}_t) = p(\mathbf{x}_t, y)$$

It is equivalent to the condition enhancement in **Classifier Guidance**!

CFG Training

$$\mathbb{E}_{t>1, q(x_t|x_0), y} [\|\hat{\epsilon}_\theta(x_t, t, y) - \epsilon_t\|^2]$$

Repeat:

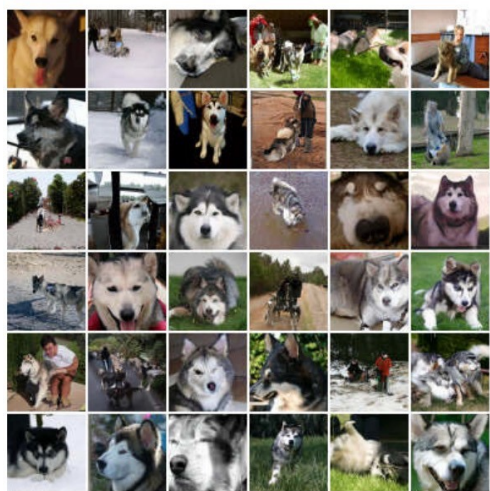
1. Take a random x_0 and its condition y (text prompt or class label).
2. Sample $t \sim \mathcal{U}(\{1, \dots, T\})$.
3. Sample $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
4. Compute $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t$.
5. Randomly dropout condition y (with null token ϕ)
6. Take gradient decent step on $\nabla_\theta \|\hat{\epsilon}_\theta(x_t, t) - \epsilon_t\|^2$.

讓模型 $\hat{\epsilon}_\theta$ 同時學習 conditional distribution $p(x_t|y)$ 和 unconditional distribution $p(x_t)$

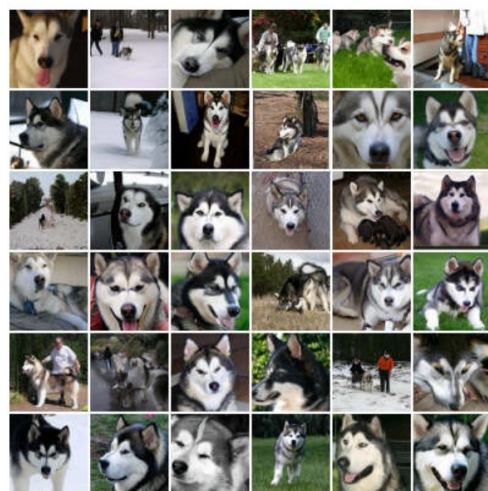
CFG Reverse Process (Generation)

1. Sample $\mathbf{x}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. **Choose condition y and guidance scale w (通常 $w \in [1.0, 10.0]$).**
3. For $t = T, \dots, 1$, repeat:
 1. Compute unconditional noise: $\hat{\epsilon}_{\text{uncond}} = \hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$
 2. Compute conditional noise: $\hat{\epsilon}_{\text{cond}} = \hat{\epsilon}_{\theta}(\mathbf{x}_t, y, t)$
 3. Apply CFG combination: $\tilde{\epsilon}_{\theta}(\mathbf{x}_t, y, t) = (1 + \omega)\hat{\epsilon}_{\theta}(\mathbf{x}_t, y, t) - \omega\hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$
 4. Compute $\tilde{\mu} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \tilde{\epsilon}_{\theta}(\mathbf{x}_t, y, t) \right)$.
 5. Sample $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
 6. Compute $\mathbf{x}_{t-1} = \tilde{\mu} + \tilde{\sigma} \mathbf{z}_t$.

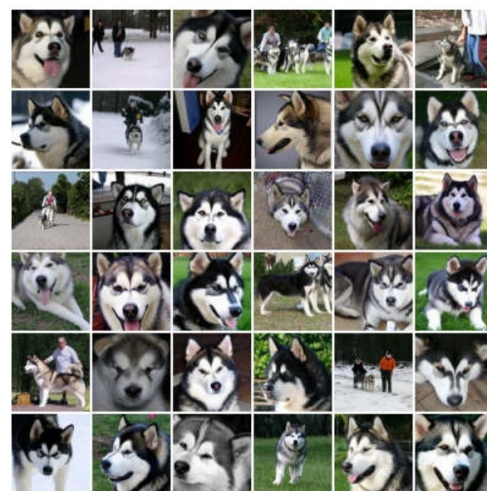
Classifier-Free Guidance (CFG)



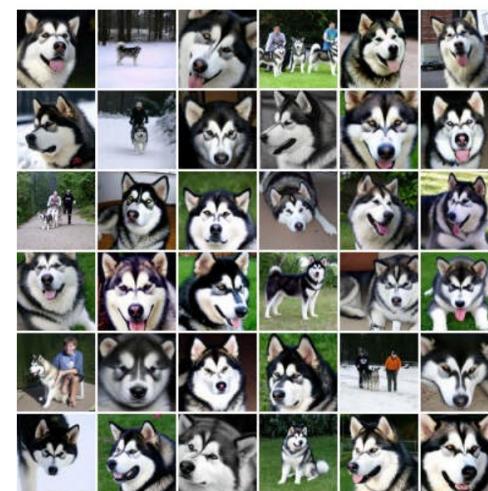
(a) $w = 0$



(b) $w = 1$



(c) $w = 2$



(d) $w = 4$



Classifier-Free Guidance (CFG)

Model		FID (↓)
Classifier Guidance	BigGAN-deep, max IS (Brock et al., 2019)	25
	BigGAN-deep (Brock et al., 2019)	5.7
	CDM (Ho et al., 2021)	3.52
	LOGAN (Wu et al., 2019)	3.36
	ADM-G (Dhariwal & Nichol, 2021)	2.97
	Ours	$T = 128 / 256 / 1024$
	$w = 0.0$	8.11 / 7.27 / 7.22
	$w = 0.1$	5.31 / 4.53 / 4.5
	$w = 0.2$	3.7 / 3.03 / 3
	$w = 0.3$	3.04 / 2.43 / 2.43
	$w = 0.4$	3.02 / 2.49 / 2.48
	$w = 0.5$	3.43 / 2.98 / 2.96
	$w = 0.6$	4.09 / 3.76 / 3.73
	$w = 0.7$	4.96 / 4.67 / 4.69
	$w = 0.8$	5.93 / 5.74 / 5.71
	$w = 0.9$	6.89 / 6.8 / 6.81
	$w = 1.0$	7.88 / 7.86 / 7.8
	$w = 2.0$	15.9 / 15.93 / 15.75
	$w = 3.0$	19.77 / 19.77 / 19.56
	$w = 4.0$	21.55 / 21.53 / 21.45

ImageNet Results

Classifier-Free Guidance (CFG)

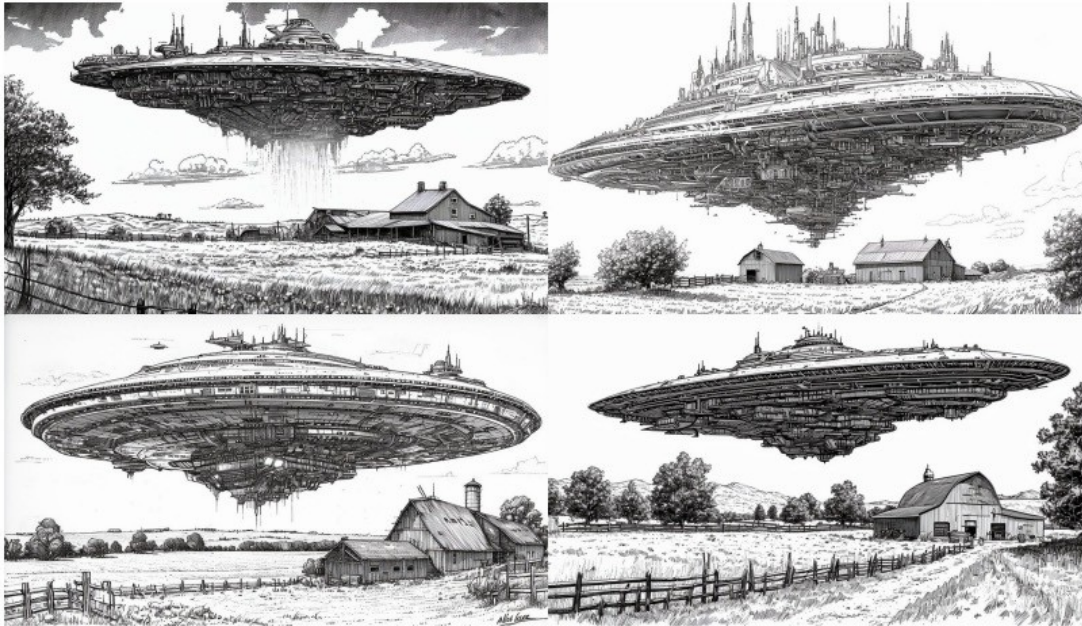
- (+) Very easy to implement.
- (+) Versatile; can be used not only for labels but also for any additional information (e.g., text descriptions).
- (-) The noise predictor needs to be evaluated twice in the generation process.
- (-) 需要準備有 condition (如類別或 text prompt) 的影像訓練資料 (使用 BLIP-2 等 captioning models 不難準備)

Classifier-Free Guidance (CFG)

- (+) Very easy to implement.
- (+) Versatile; can be used not only for labels but also for any additional information (e.g., text descriptions).
- (-) The noise predictor needs to be evaluated twice in the generation process.
- (-) 需要準備有 condition (如類別或 text prompt) 的影像訓練資料 (使用 BLIP-2 等 captioning models 不難準備)

Example: Text-to-Image (T2I)

Stable Diffusion 3



detailed pen and ink drawing of a massive complex alien space ship above a farm in the middle of nowhere.



photo of a bear wearing a suit and top hat in a river in the middle of a forest holding a sign that says "I cant bear it".

Example: Text-to-Image (T2I)

Stable Diffusion 3



tilt shift aerial photo of a cute city made of sushi on a wooden table in the evening.



dark high contrast render of a psychedelic tree of life illuminating dust in a mystical cave.

Example: Text-to-Image (T2I)

Stable Diffusion 3



an anthropomorphic fractal person behind the counter at a fractal themed restaurant.



beautiful oil painting of a steamboat in a river in the afternoon. On the side of the river is a large brick building with a sign on top that says SD3.

Example: Text-to-Image (T2I)

Stable Diffusion 3

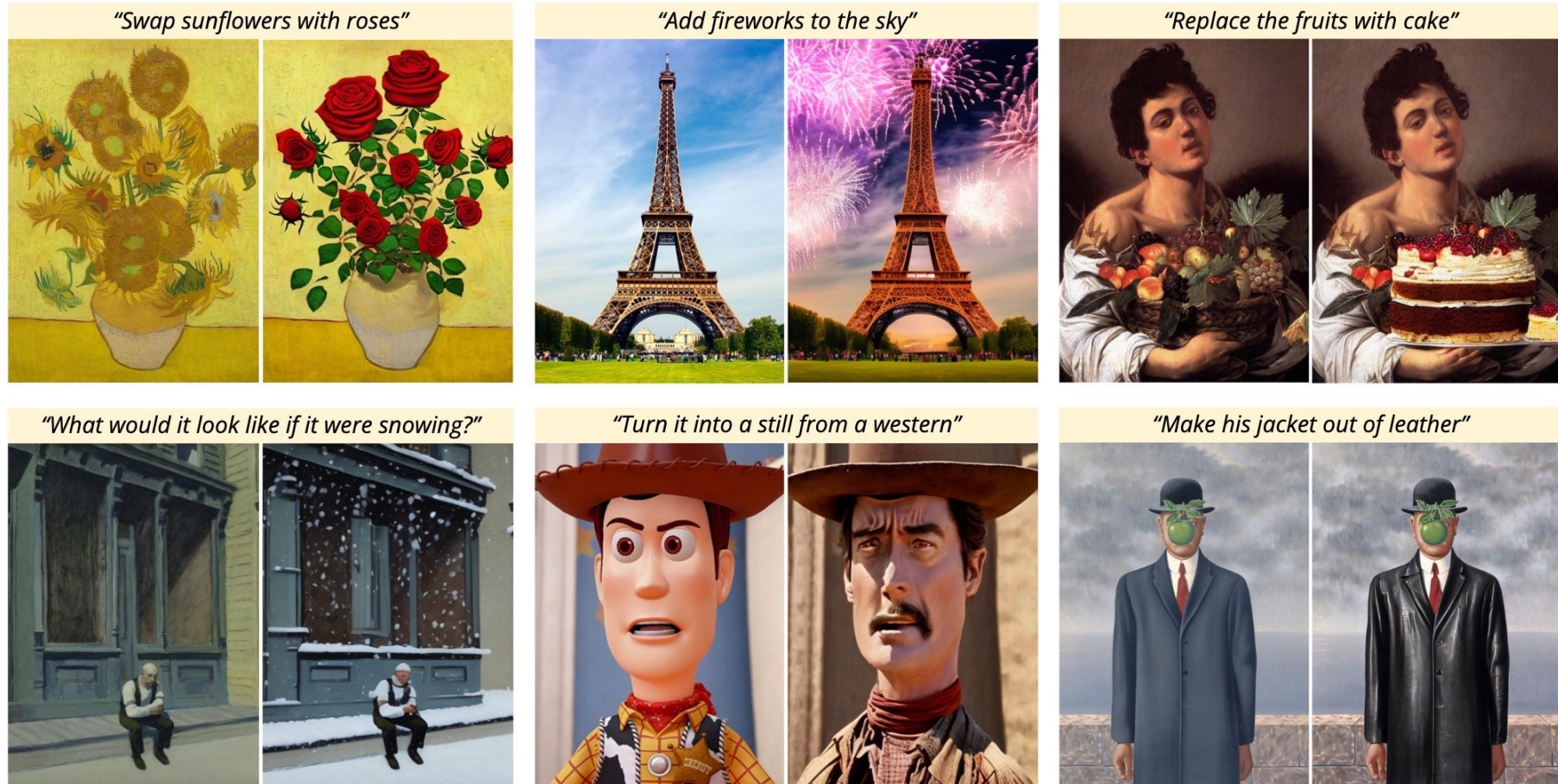


an anthropomorphic pink donut with a mustache and cowboy hat standing by a log cabin in a forest with an old 1970s orange truck in the driveway

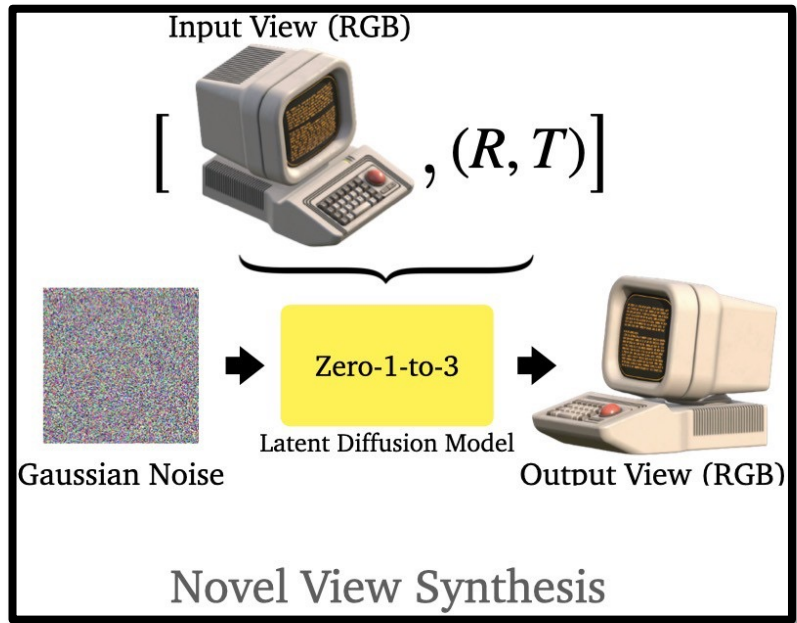


fox sitting in front of a computer in a messy room at night. On the screen is a 3d modeling program with a line render of a zebra.

Example: Image-to-Image



Example: Zero 1-to-3



Down 30° Left: 90°



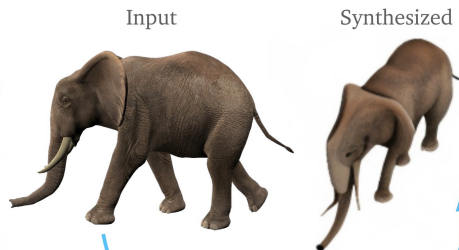
Up 45° Right 60°



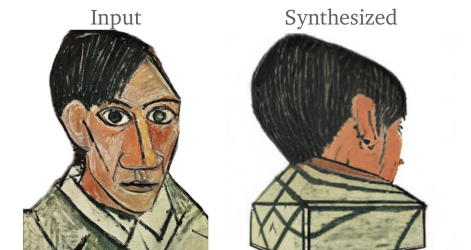
Up: 90°



Down: 25° Right: 95°



Up: 45° Left: 60°



Left: 120°

Audio-to-Image



Negative Prompt

Prompt: Portrait of zimby anton fadeev cyborg propaganda poster

[NPW] Weight: 0.0

[NPW] Weight: 0.25

[NPW] Weight: 0.5

[NPW] Weight: 0.75

[NPW] Weight: 1.0



[—] Male

[NPW] Weight: 0.0

[NPW] Weight: 0.25

[NPW] Weight: 0.5

[NPW] Weight: 0.75

[NPW] Weight: 1.0



[—] Female

Negative Prompt

$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, y, t) = (1 + \omega)\hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}, t) - \omega\hat{\epsilon}_{\theta}(\mathbf{x}_t, \phi, t)$$



$$\tilde{\epsilon}_{\theta}(\mathbf{x}_t, y, t) = (1 + \omega)\hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}_+, t) - \omega\hat{\epsilon}_{\theta}(\mathbf{x}_t, \mathbf{y}_-, t)$$

Positive prompt

Negative prompt

Latent Diffusion

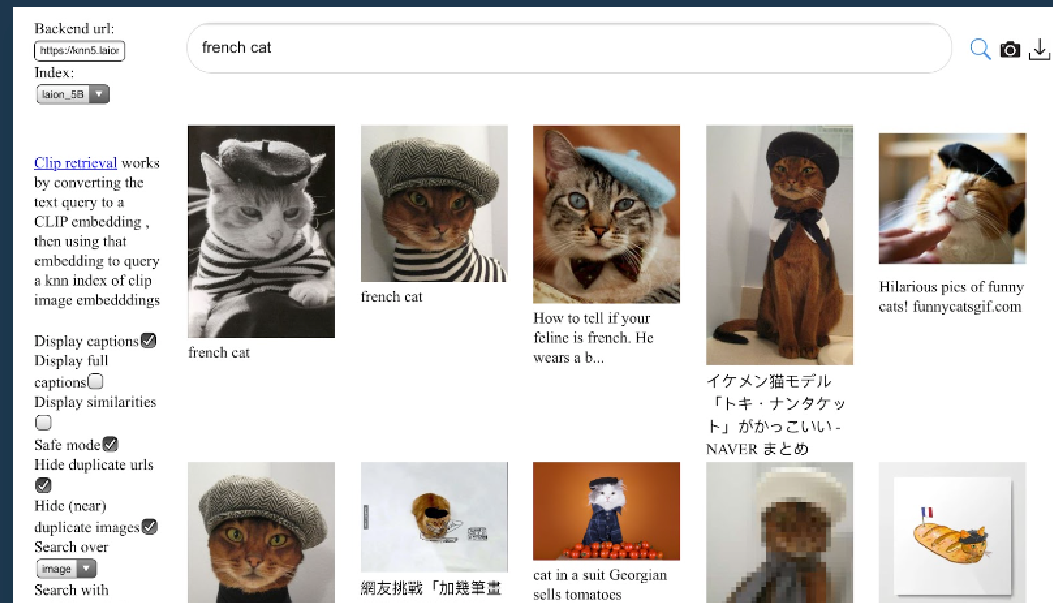
Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022.

LAION-5B: A NEW ERA OF OPEN LARGE-SCALE MULTI-MODAL DATASETS

by: Romain Beaumont, 31 Mar, 2022

We present a dataset of **5.85 billion** CLIP-filtered image-text pairs, 14x bigger than LAION-400M, previously the biggest openly accessible image-text dataset in the world - see also our [NeurIPS2022 paper](#)

Authors: Christoph Schuhmann, Richard Vencu, Romain Beaumont, Theo Coombes, Cade Gordon, Aarush Katta, Robert Kaczmarczyk, Jenia Jitsev



High-Resolution Images



High-Resolution Images

- Training: Hundreds of GPU days (e.g. 150 - 1000 V100 days)
- Generation: Generating 50k samples takes approximately 5 days on a single A100 GPU.
 - 使用單張 A100 生成高解析度的一張影像需要 8-9 秒

How to reduce the computation time?

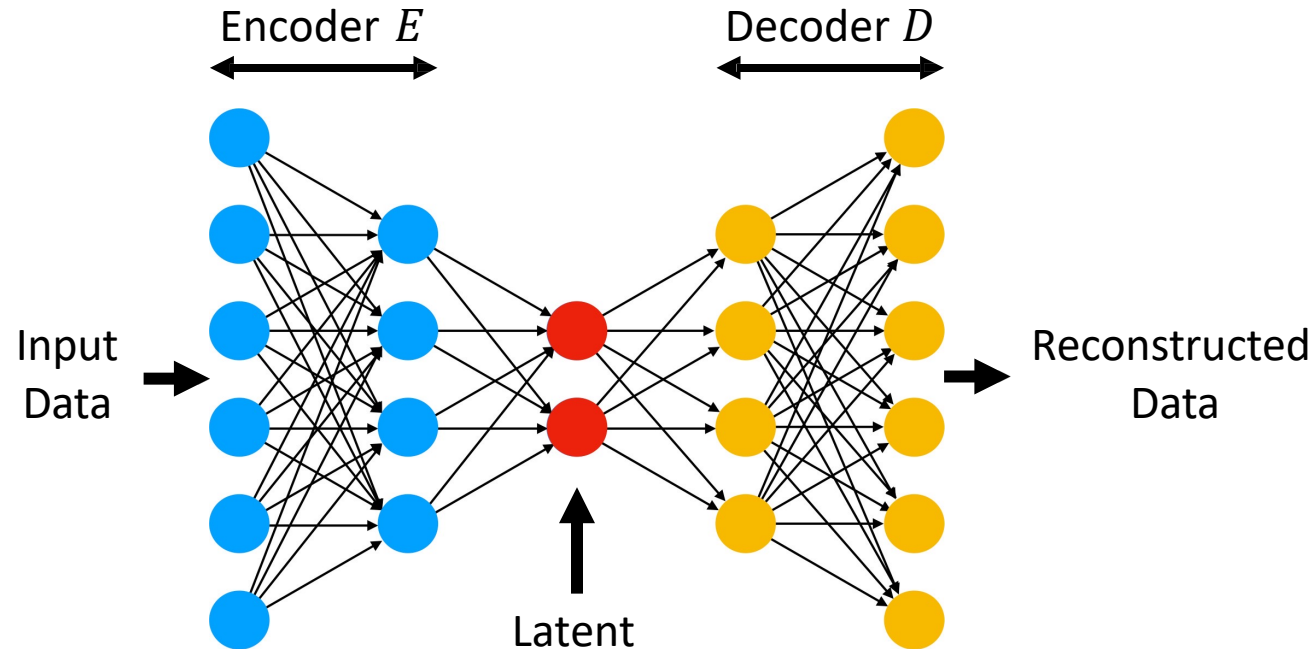
→ **Image compression.**

(解析度上的，不是影像壓縮 codec)

Image Compression

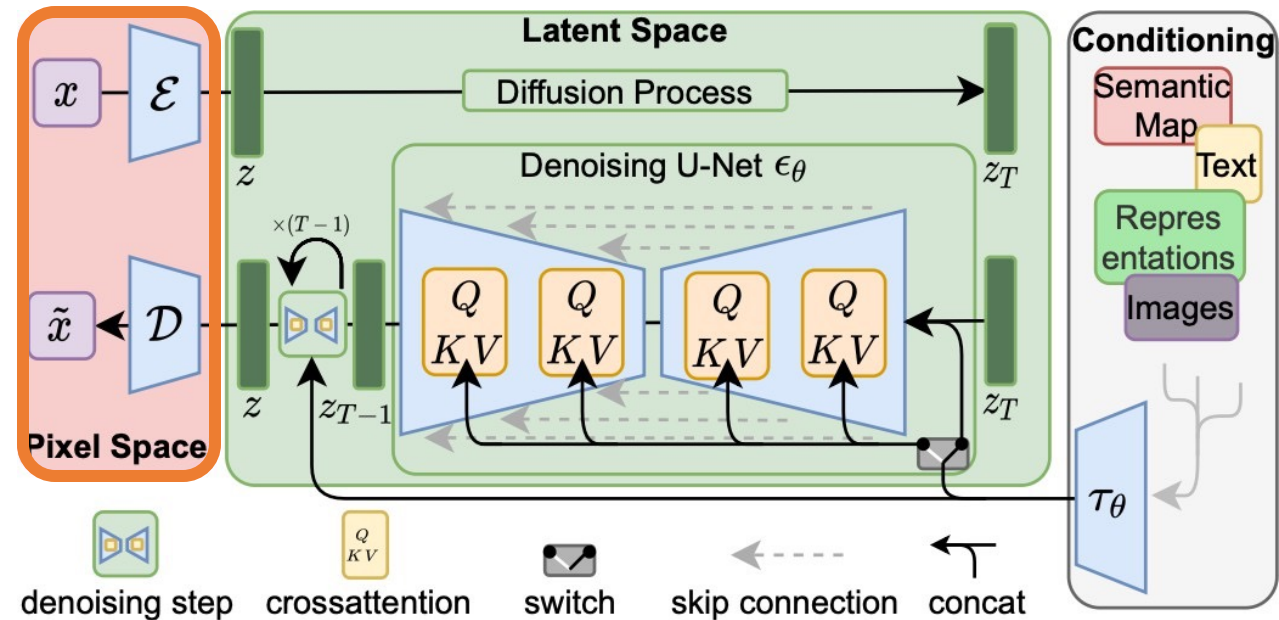
Pretrain an autoencoder (with KL-divergence regularization loss) to decrease the resolution from, for example,

$$512 \times 512 \times 3 \Rightarrow 64 \times 64 \times 4.$$



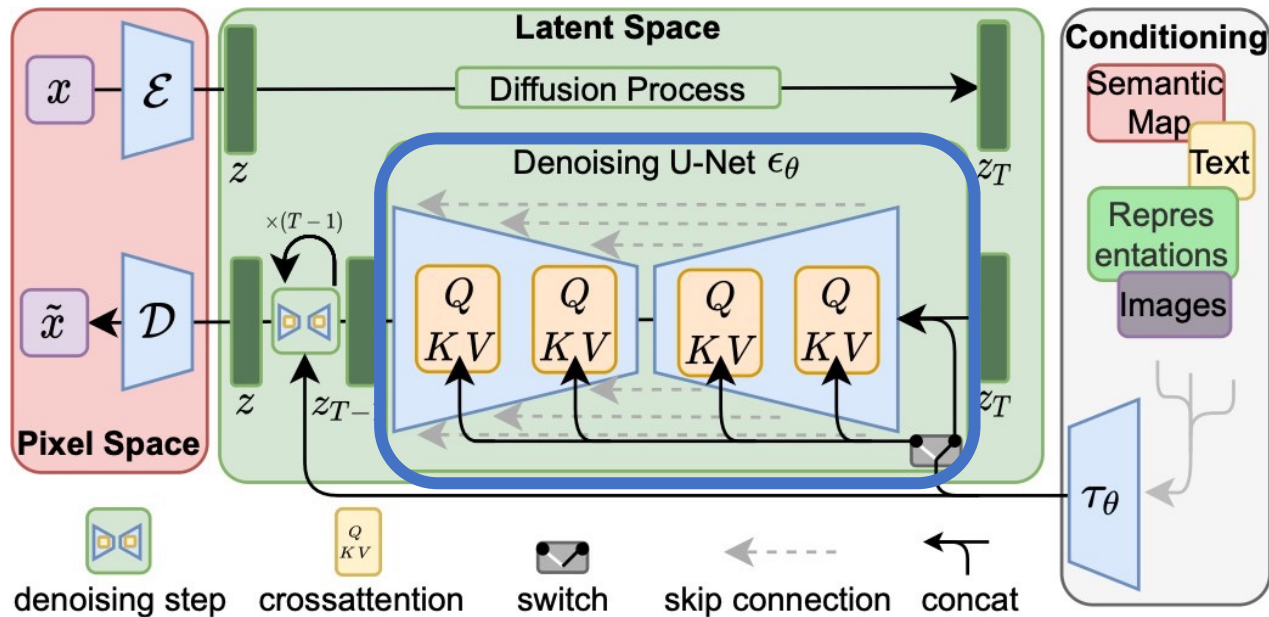
Latent Diffusion

- Image encoder/decoder are **pretrained**.

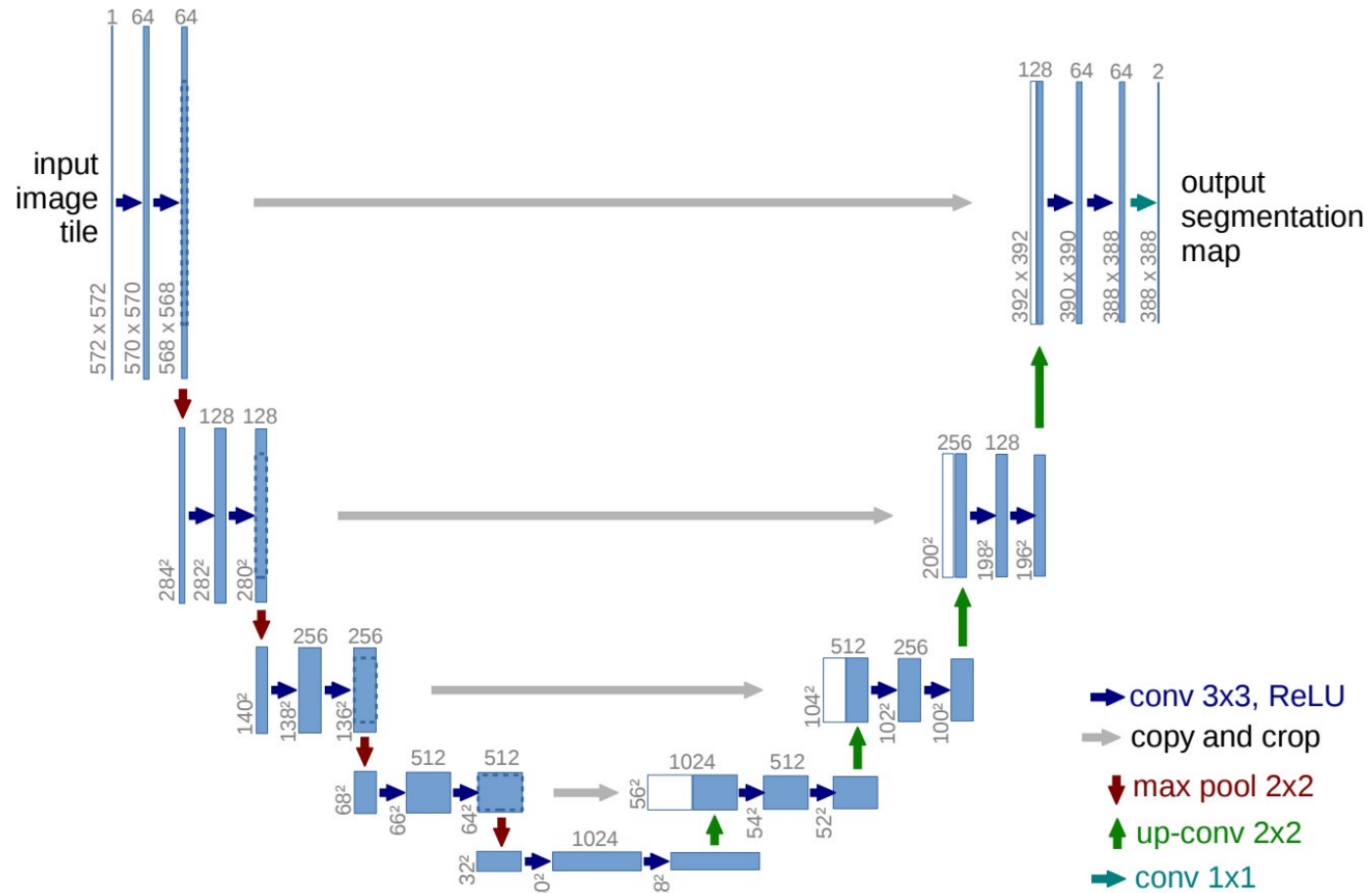


Latent Diffusion

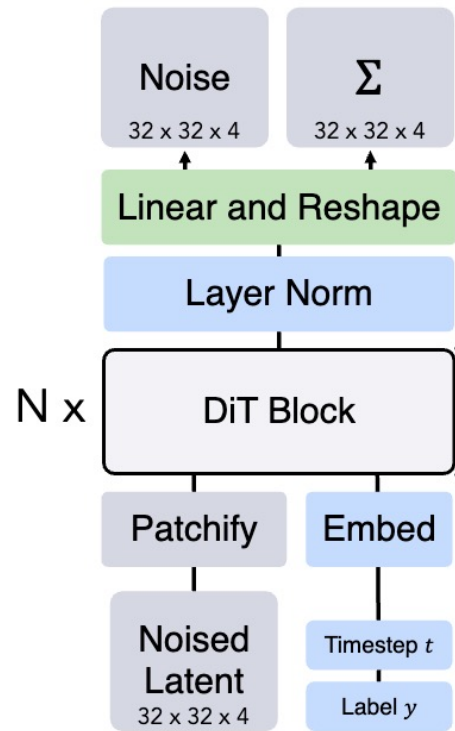
- The **noise predictor** is trained in the **latent space**.
- For noise prediction, earlier models use **U-Net** with **attention module** at each layer.



Latent Diffusion



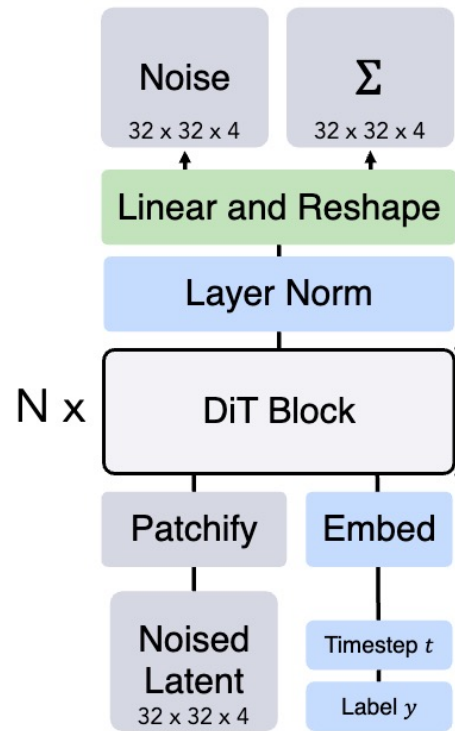
Latent Diffusion



Latent Diffusion Transform

把 U-Net 換成更強的 Transformer 就變成 Diffusion Transformer (DiT)

Latent Diffusion

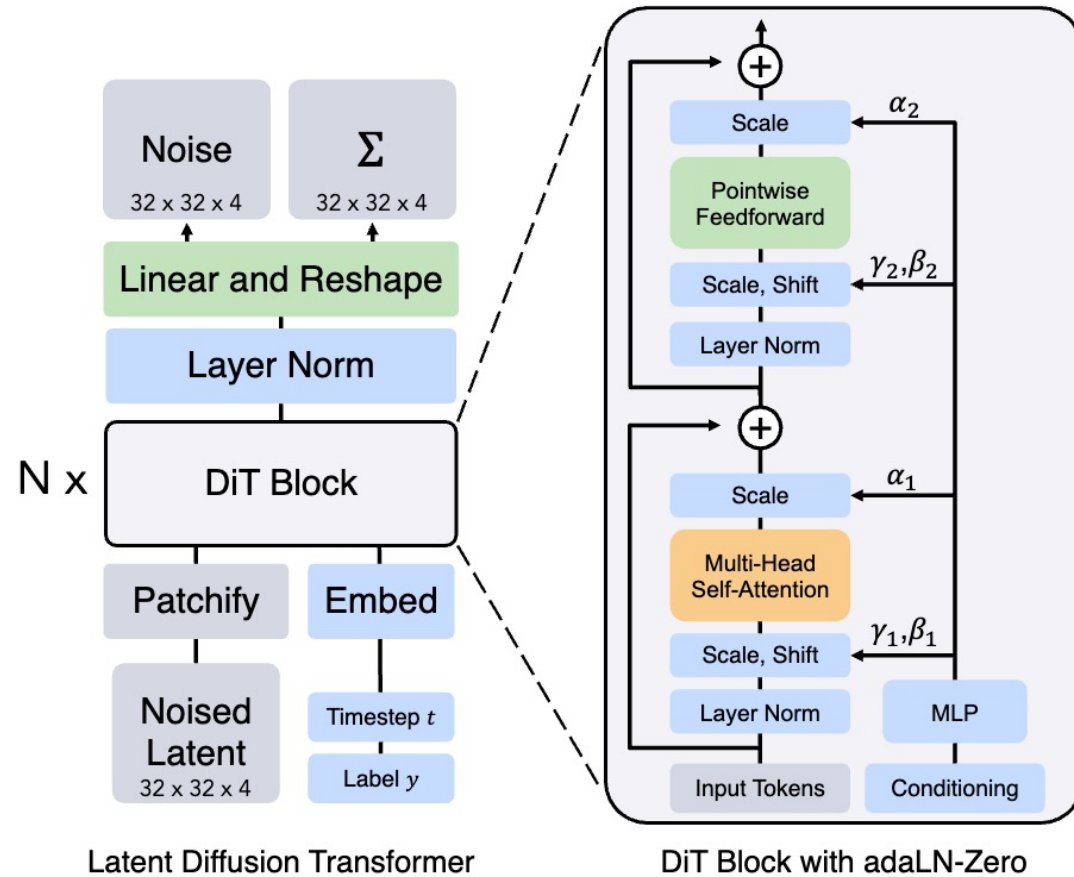


Latent Diffusion Transform

Q. How to inject conditioning (timestep t , text, ...)

把 U-Net 換成更強的 Transformer 就變成 Diffusion Transformer (DiT)

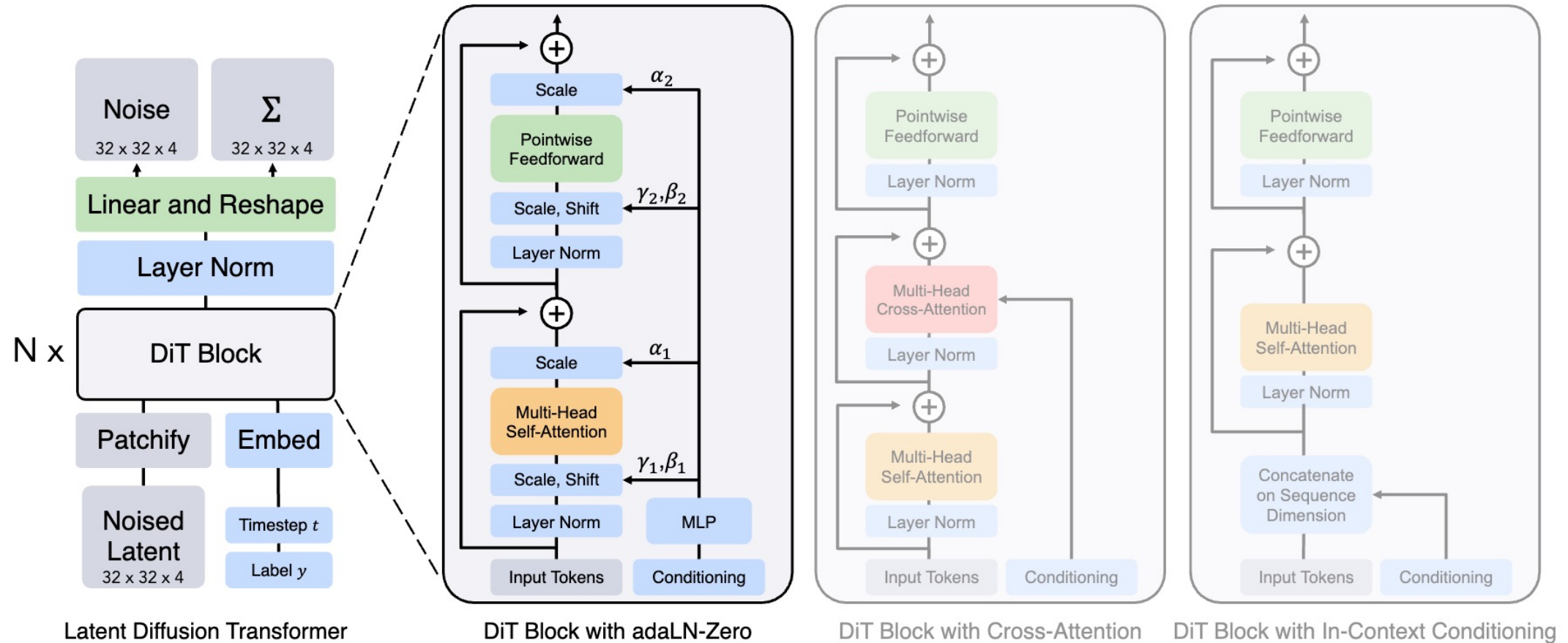
Latent Diffusion



Predict scale/shift:

Most common for diffusion timestep t

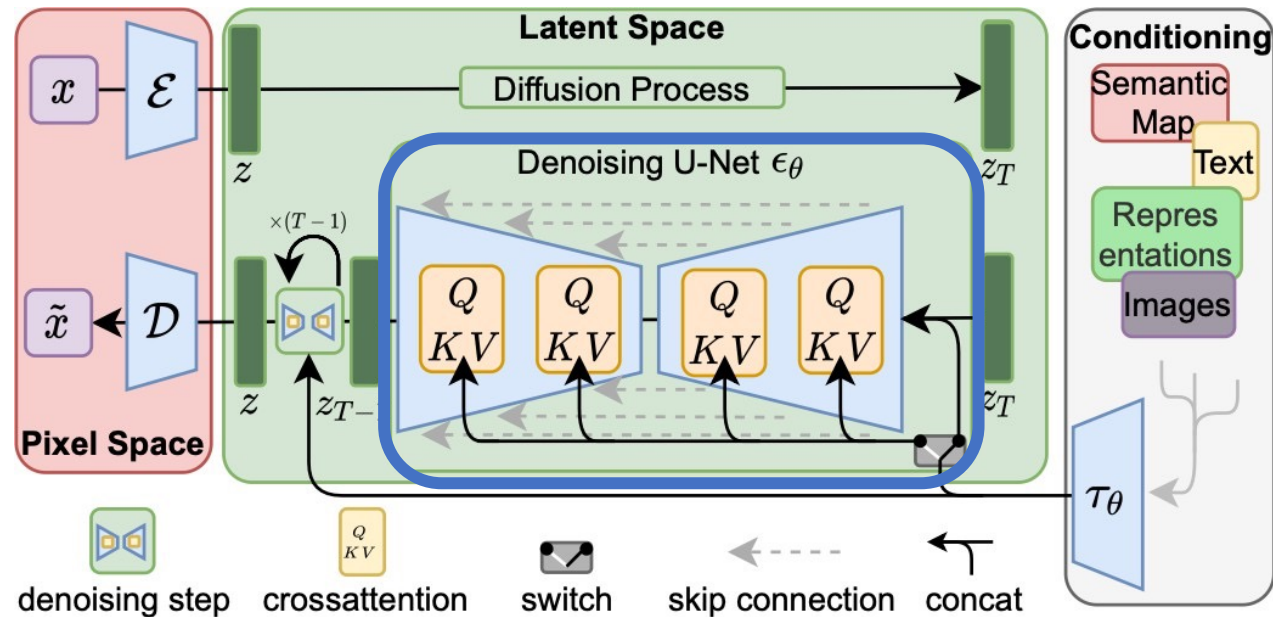
Latent Diffusion



Cross-Attention / Joint Attention:
Common for text, image, etc conditioning

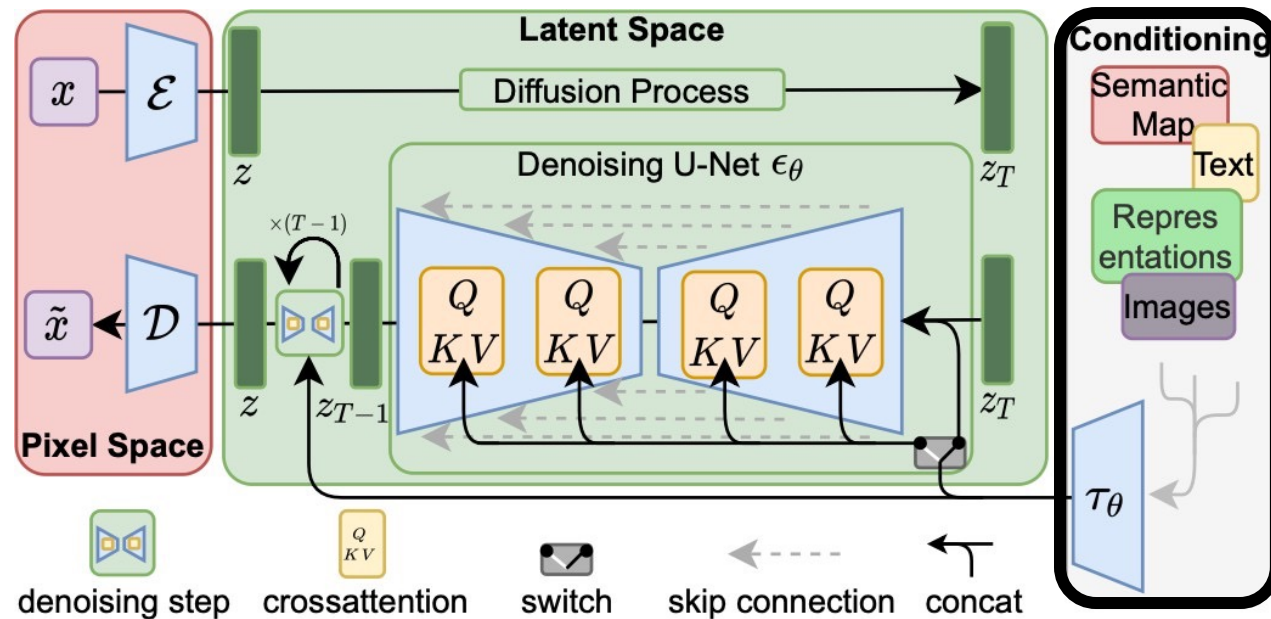
Latent Diffusion

- The **noise predictor** is trained in the **latent space**.
- For noise prediction, earlier models use **U-Net** with **attention module** at each layer.



Latent Diffusion

- The input **condition** is incorporated into the **cross-attention** module.



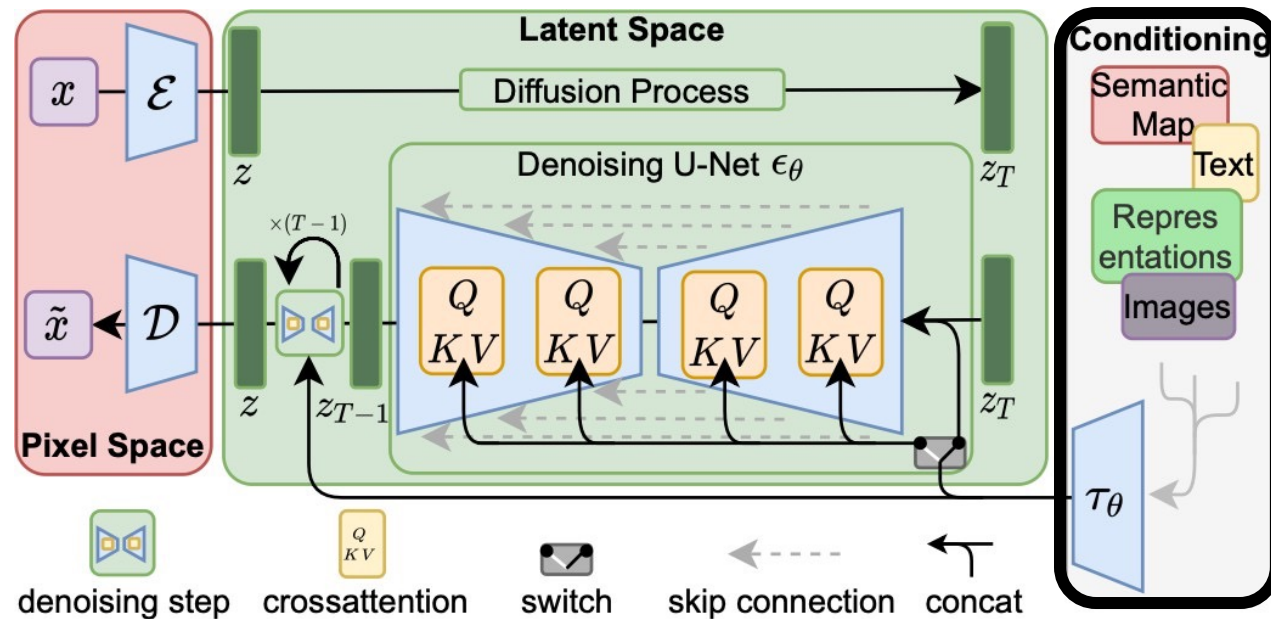
Latent Diffusion

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V$$

- Query Q : The output of each U-Net layer.
- Key K and Value V : The outputs from the input condition encoder.

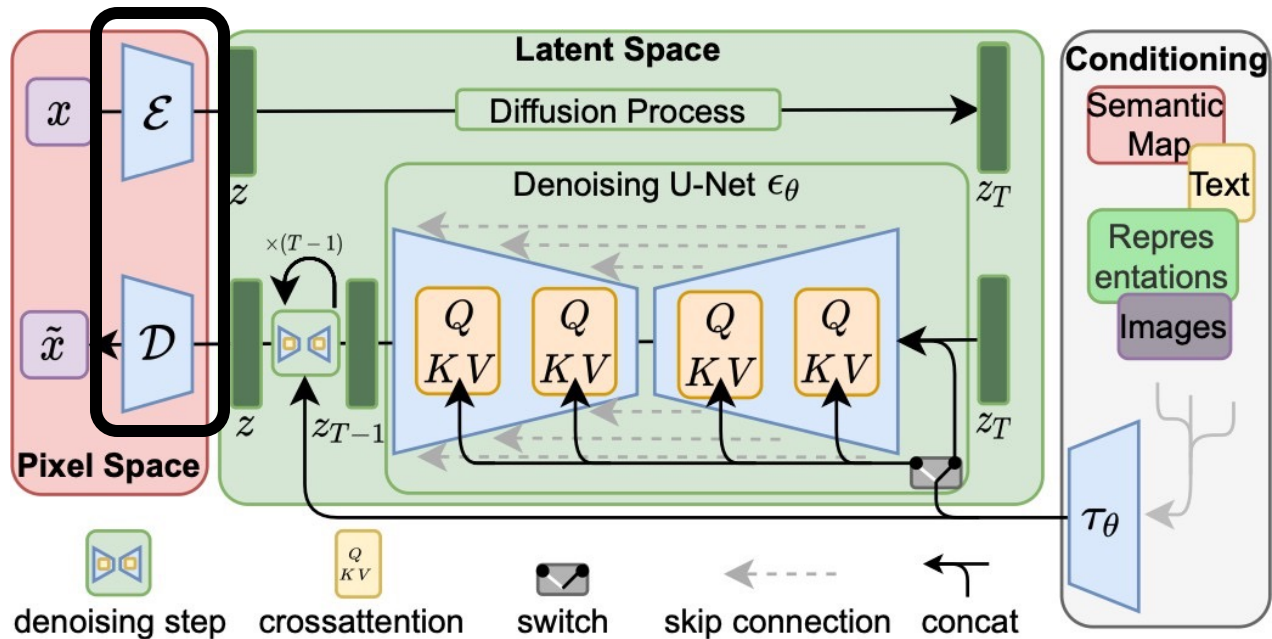
Latent Diffusion

- The input **condition** is incorporated into the **cross-attention** module.



Latent Diffusion

Q. Why Variational Autoencoder (VAE) instead of Autoencoder (AE)?



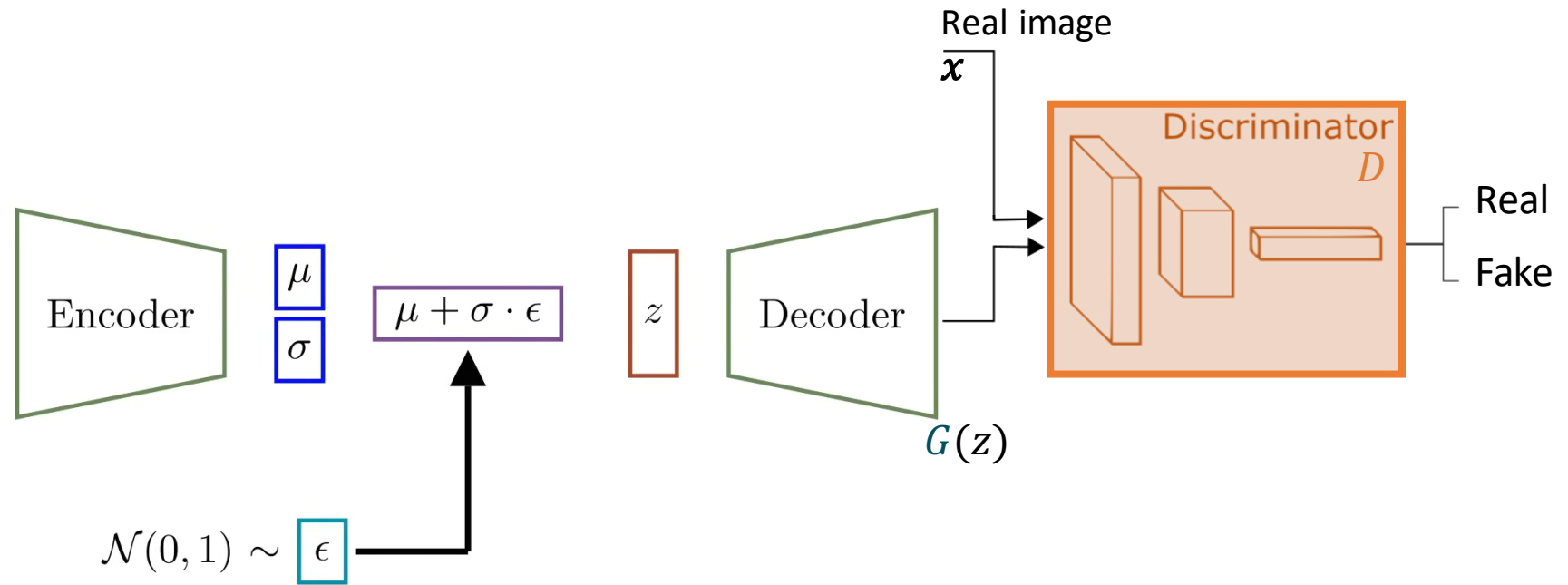
Latent Diffusion

A.

- VAE 可以 regularize latent space follow 標準正態分佈，增加訓練穩定性（Diffusion 模型需要預測 noise，如果輸入範圍不穩定，訓練會很困難）
- VAE 的 latent space 更連續平滑，相似圖片的 latent 距離相近，中間點的 latent 對應合理的圖片，Diffusion 的 denoise 路徑更自然
- Trade-off
 - KL 權重太小 -> latent 不規則，Diffusion 訓練困難
 - KL 權重太大 -> 重建 quality 下降，影像細節不見

Latent Diffusion

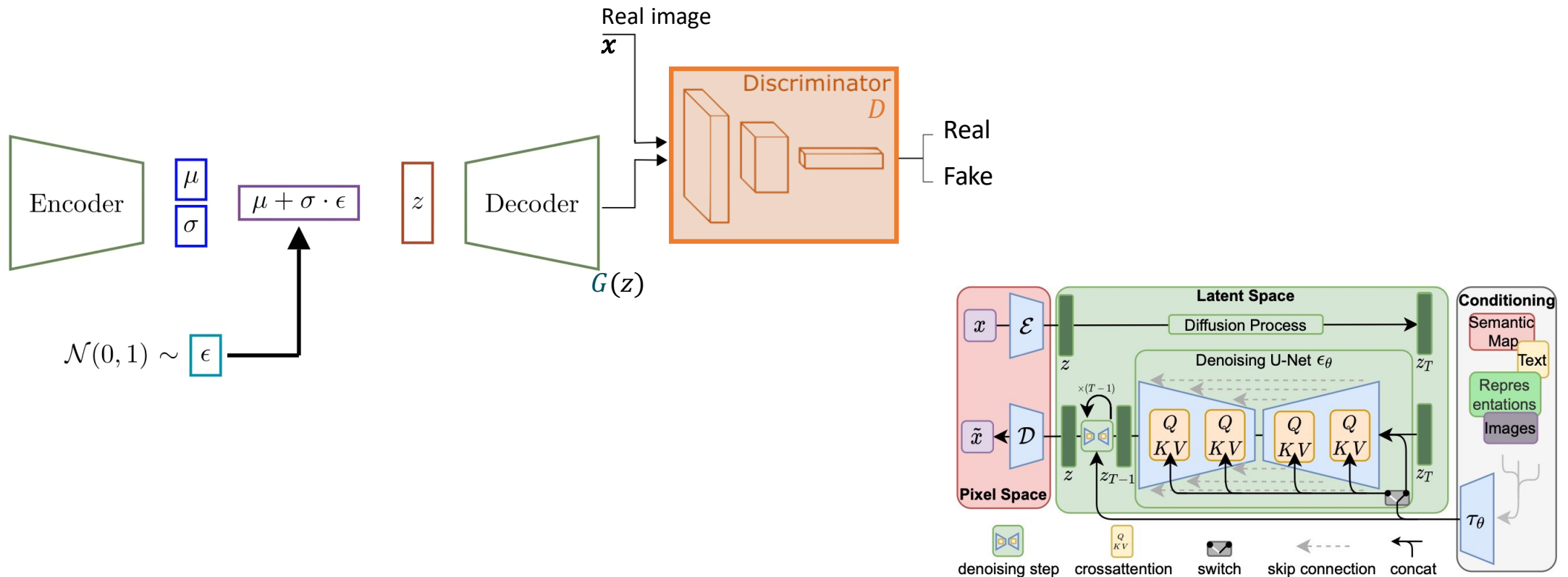
A. Make it a GAN!



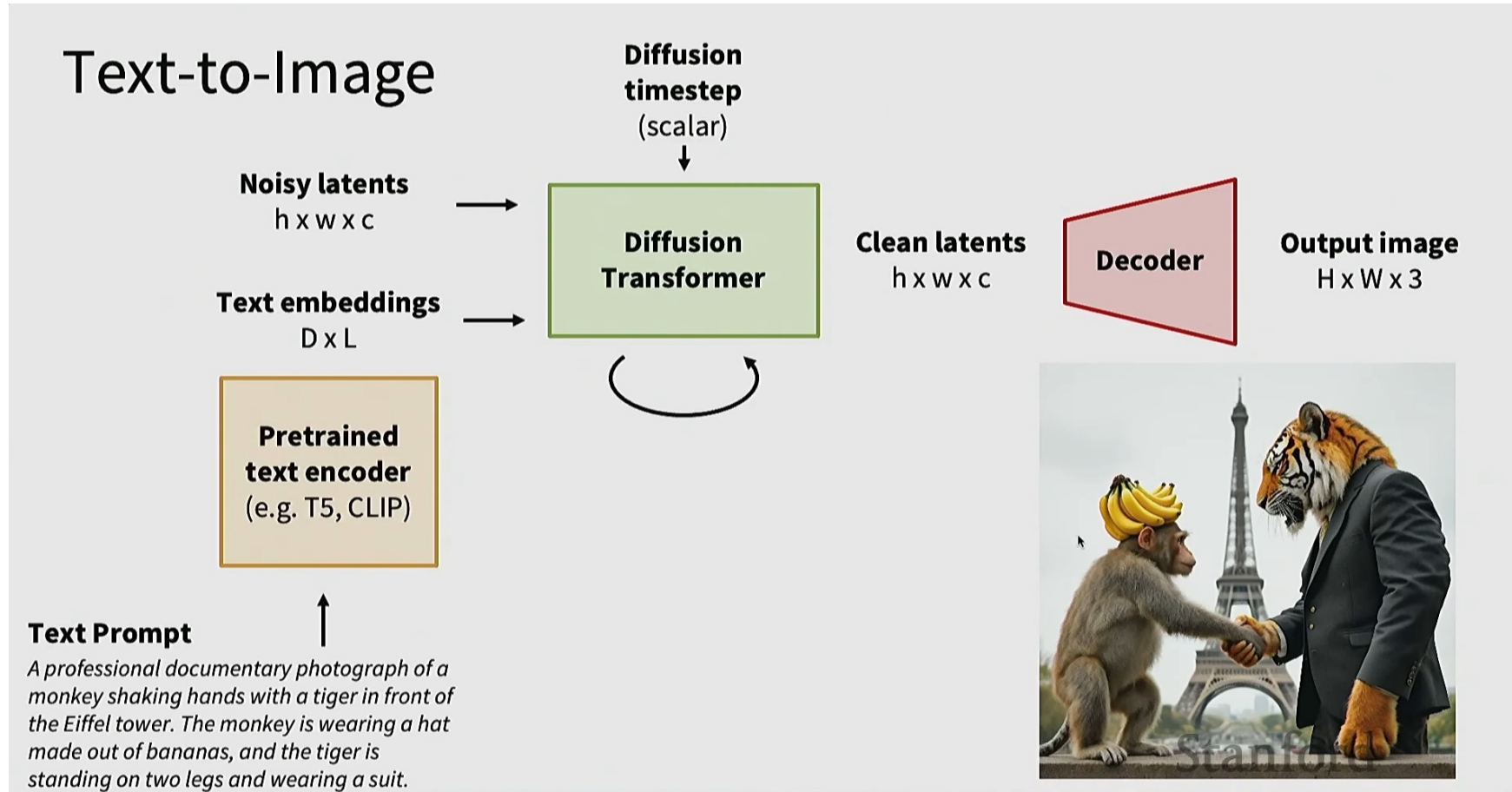
Solution: Add a discriminator!

Latent Diffusion

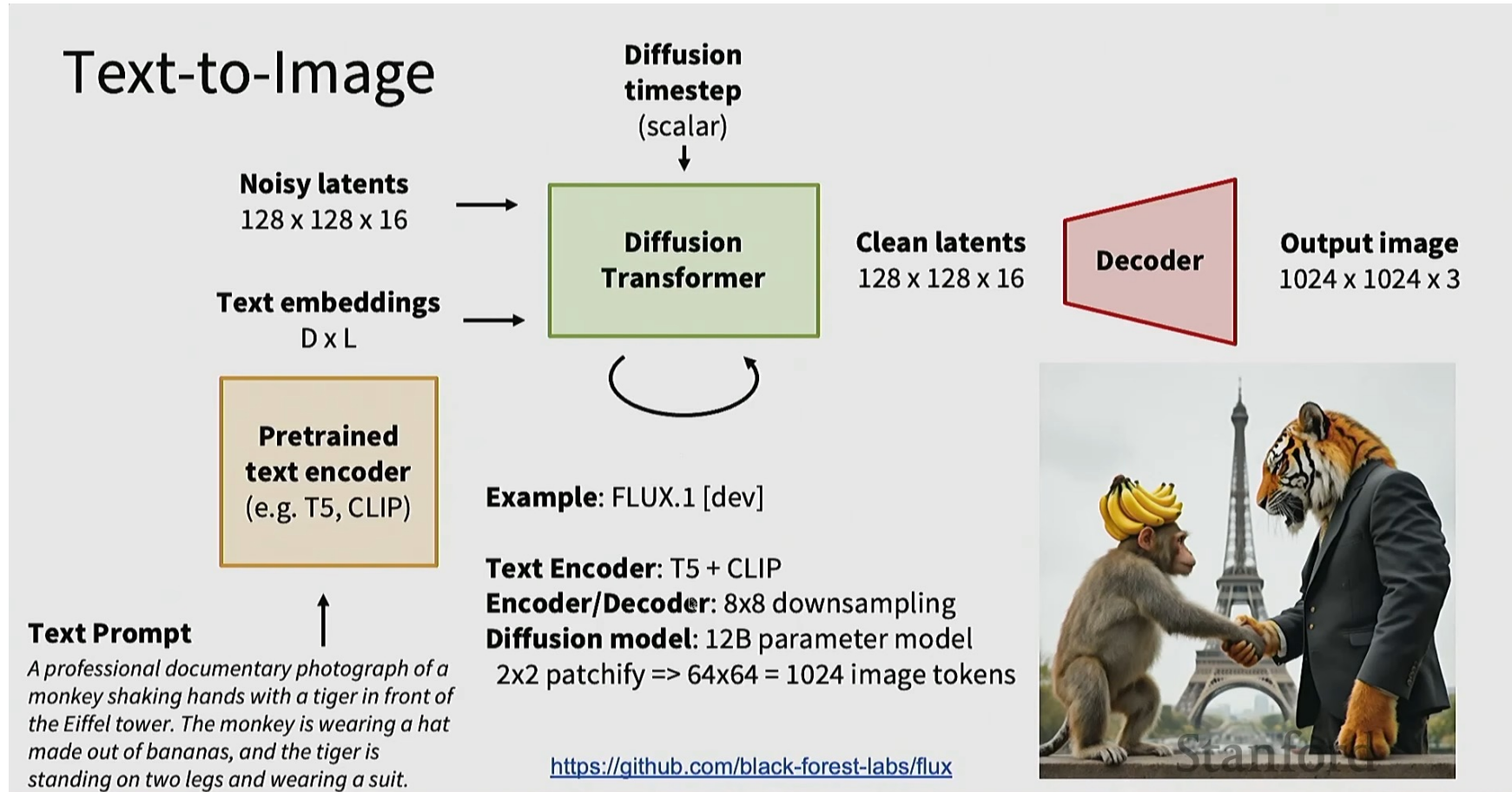
Modern Latent Diffusion Model pipelines use **VAE** + **GAN** + **diffusion**!



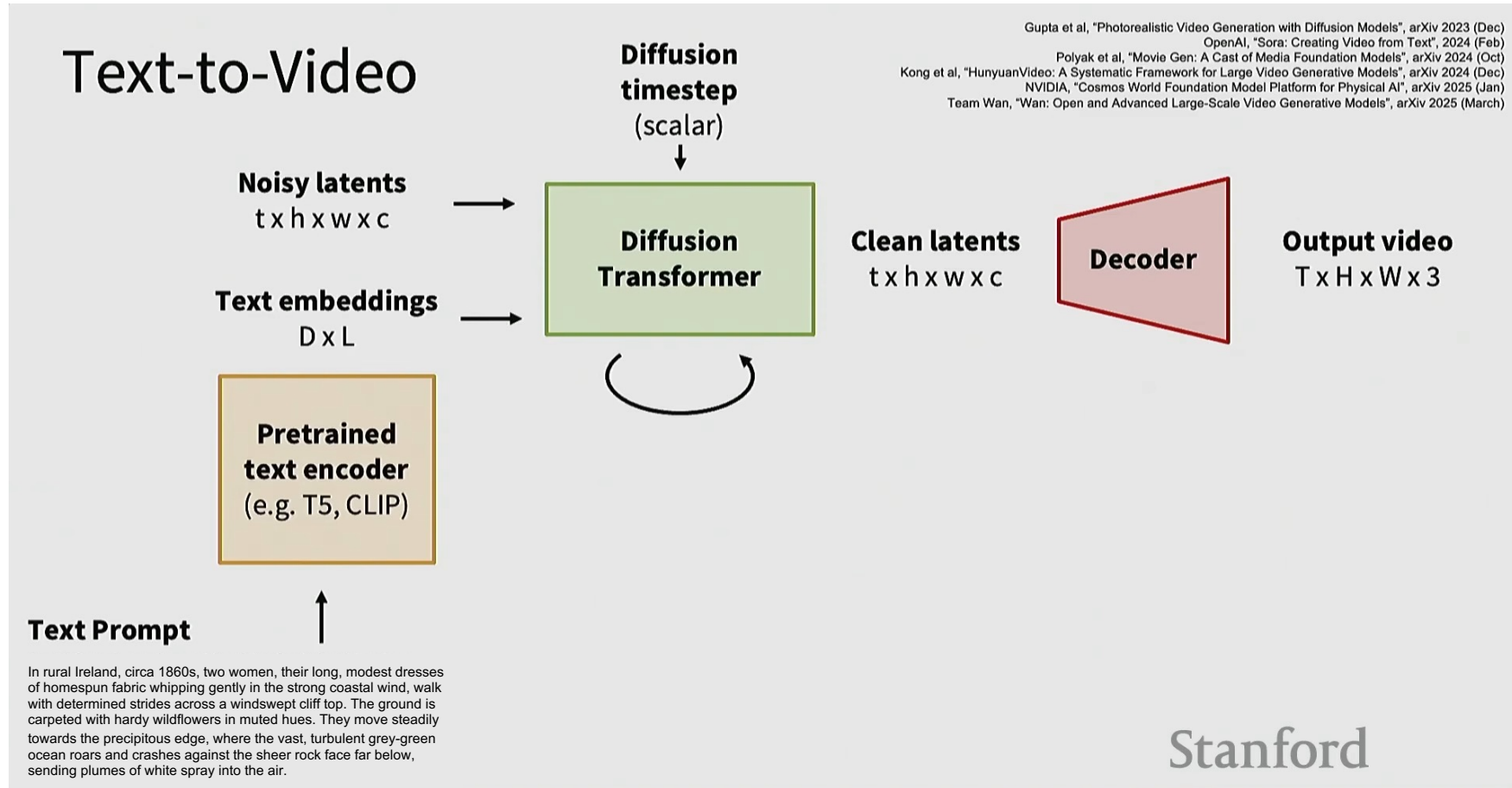
Text-to-Image



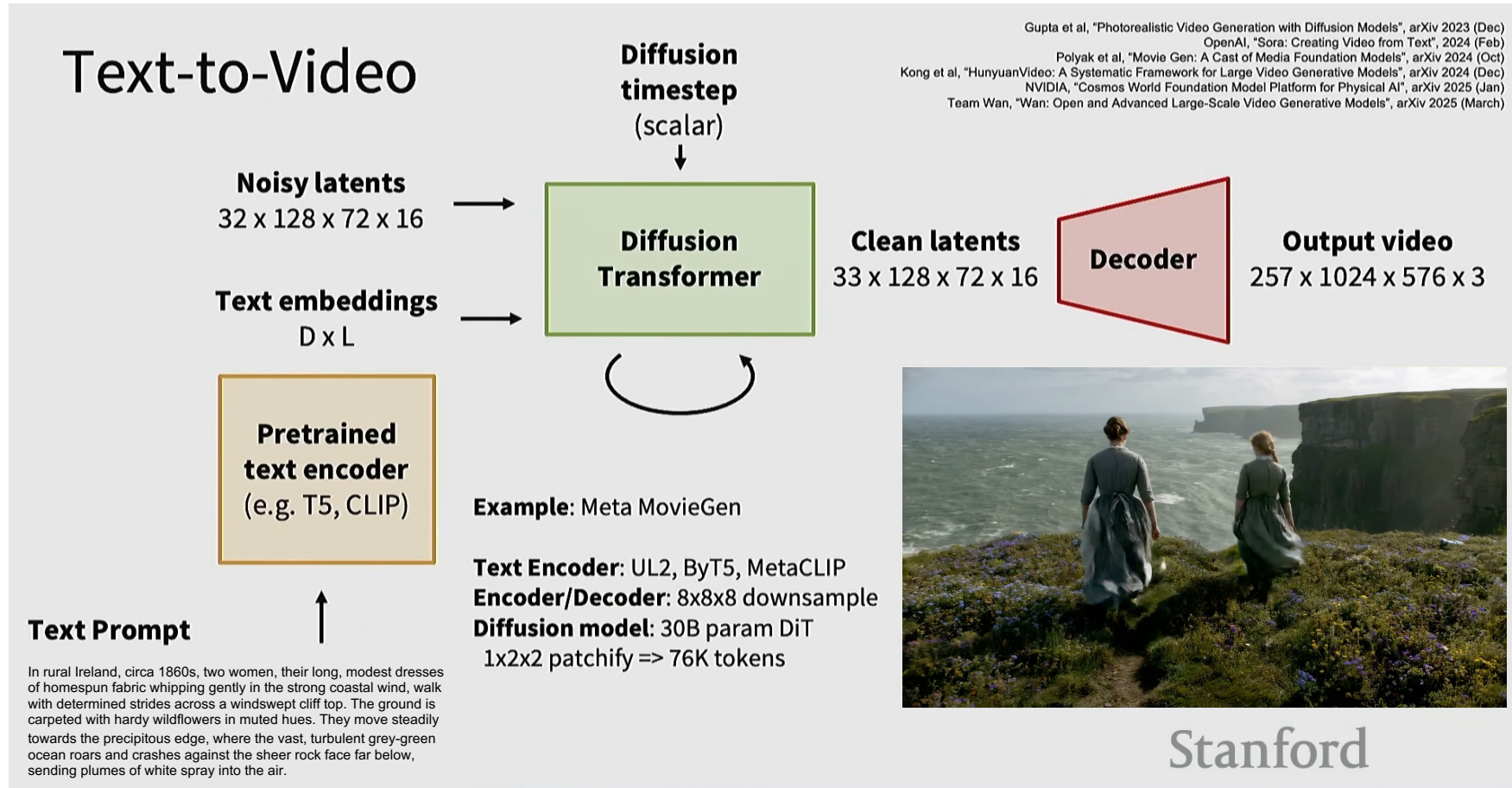
Text-to-Image



Text-to-Video

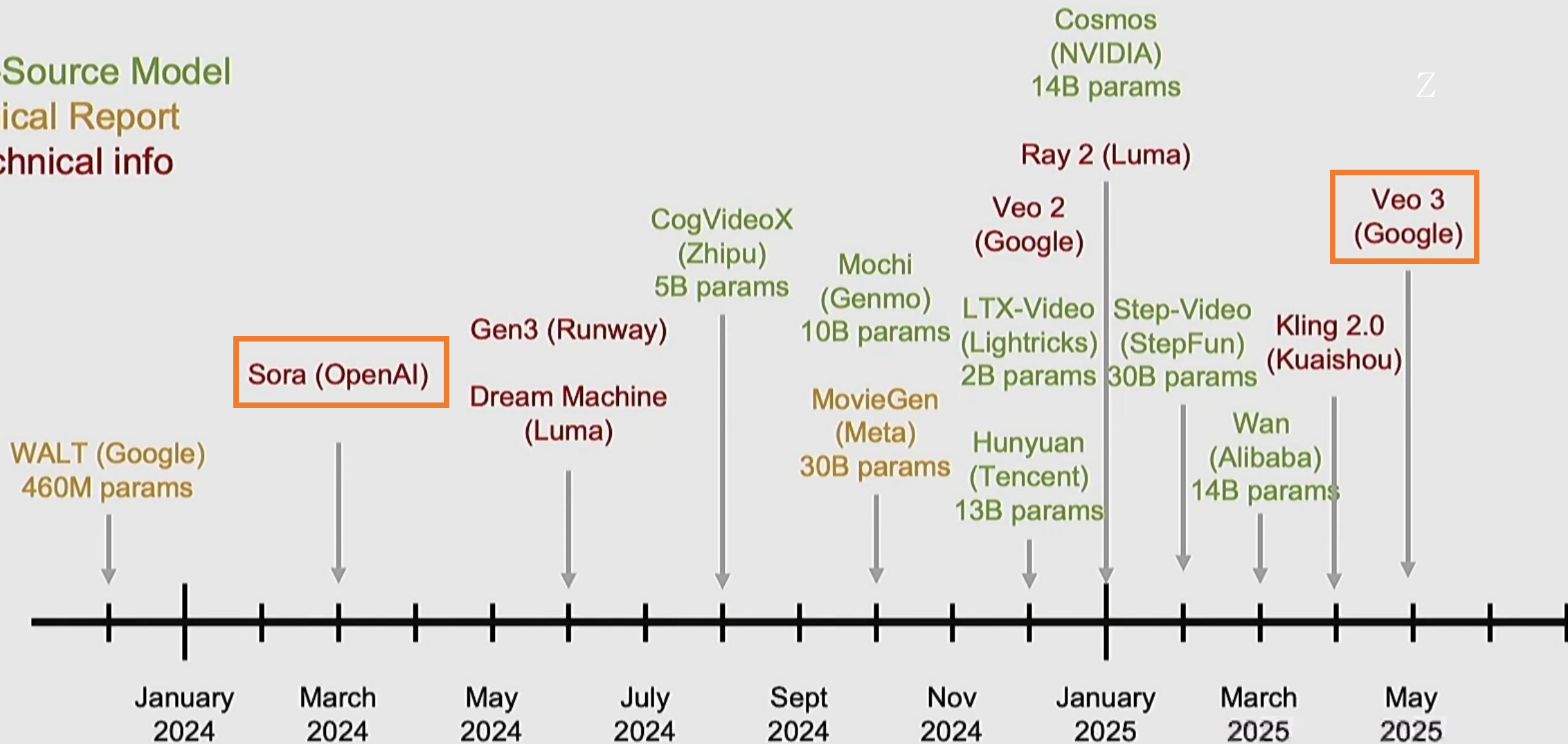


Text-to-Video



The Era of Video Diffusion Models

Open-Source Model
Technical Report
No technical info



Diffusers



license Apache-2.0

release v0.30.3

downloads/month 3M

Contributor Covenant 2.1

Follow @diffuserslib

😊 Diffusers is the go-to library for state-of-the-art pretrained diffusion models for generating images, audio, and even 3D structures of molecules. Whether you're looking for a simple inference solution or training your own diffusion models, 😊 Diffusers is a modular toolbox that supports both. Our library is designed with a focus on [usability over performance](#), [simple over easy](#), and [customizability over abstractions](#).

😊 Diffusers offers three core components:

- State-of-the-art [diffusion pipelines](#) that can be run in inference with just a few lines of code.
- Interchangeable noise [schedulers](#) for different diffusion speeds and output quality.
- Pretrained [models](#) that can be used as building blocks, and combined with schedulers, for creating your own end-to-end diffusion systems.

Latent Diffusion

Method	FID↓	IS↑	Precision↑	Recall↑	N_{params}
BigGan-deep [3]	6.95	203.6 ± 2.6	0.87	0.28	340M
ADM [15]	10.94	100.98	0.69	0.63	554M
ADM-G [15]	<u>4.59</u>	186.7	<u>0.82</u>	0.52	608M
<i>LDM-4</i> (ours)	10.56	103.49 ± 1.24	0.71	<u>0.62</u>	400M
<i>LDM-4-G</i> (ours)	3.60	247.67 ± 5.59	0.87	0.48	400M

Latent Diffusion

Method	FID↓	IS↑	Precision↑	Recall↑	N_{params}
BigGan-deep [3]	6.95	203.6 ± 2.6	0.87	0.28	340M
ADM [15]	10.94	100.98	0.69	0.63	554M
ADM-G [15]	<u>4.59</u>	186.7	<u>0.82</u>	0.52	608M
<i>LDM-4</i> (ours)	10.56	103.49 ± 1.24	0.71	<u>0.62</u>	400M
<i>LDM-4-G</i> (ours)	3.60	247.67 ± 5.59	0.87	0.48	400M

Q. 除了加速與降低參數量，為什麼 Latent Diffusion 的生成品質會比 pixel-space diffusion 更好？

Latent Diffusion

Method	FID↓	IS↑	Precision↑	Recall↑	N_{params}
BigGan-deep [3]	6.95	203.6 ± 2.6	0.87	0.28	340M
ADM [15]	10.94	100.98	0.69	0.63	554M
ADM-G [15]	<u>4.59</u>	186.7	<u>0.82</u>	0.52	608M
<i>LDM-4</i> (ours)	10.56	103.49 ± 1.24	0.71	<u>0.62</u>	400M
<i>LDM-4-G</i> (ours)	3.60	247.67 ± 5.59	0.87	0.48	400M

A. Pixel space 包含大量 noise、local texture 等不重要的冗餘，而經過 VAE 編碼後的 latent space 可以保留語義結構、去除 noise、並壓縮冗餘資訊，這讓 diffusion model 可以專注學習真正重要的結構，而不是被細節干擾

Fréchet Inception Distance (FID)

Heusel et al., GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium, NeurIPS 2017.

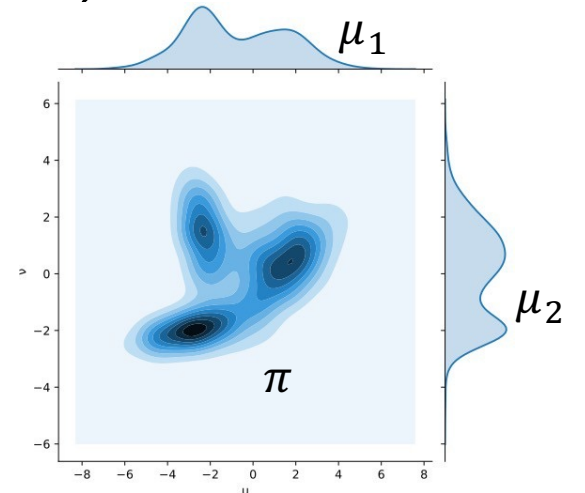
p -Wasserstein Distance

a.k.a (in a discrete setup)

Fréchet distance / Earth movers' distance

$$W_p(\mu, \nu) := \inf_{\pi \in \Gamma(\mu, \nu)} \left(\iint_{X \times X} d(x, y)^p d\pi(x, y) \right)^{1/p}$$

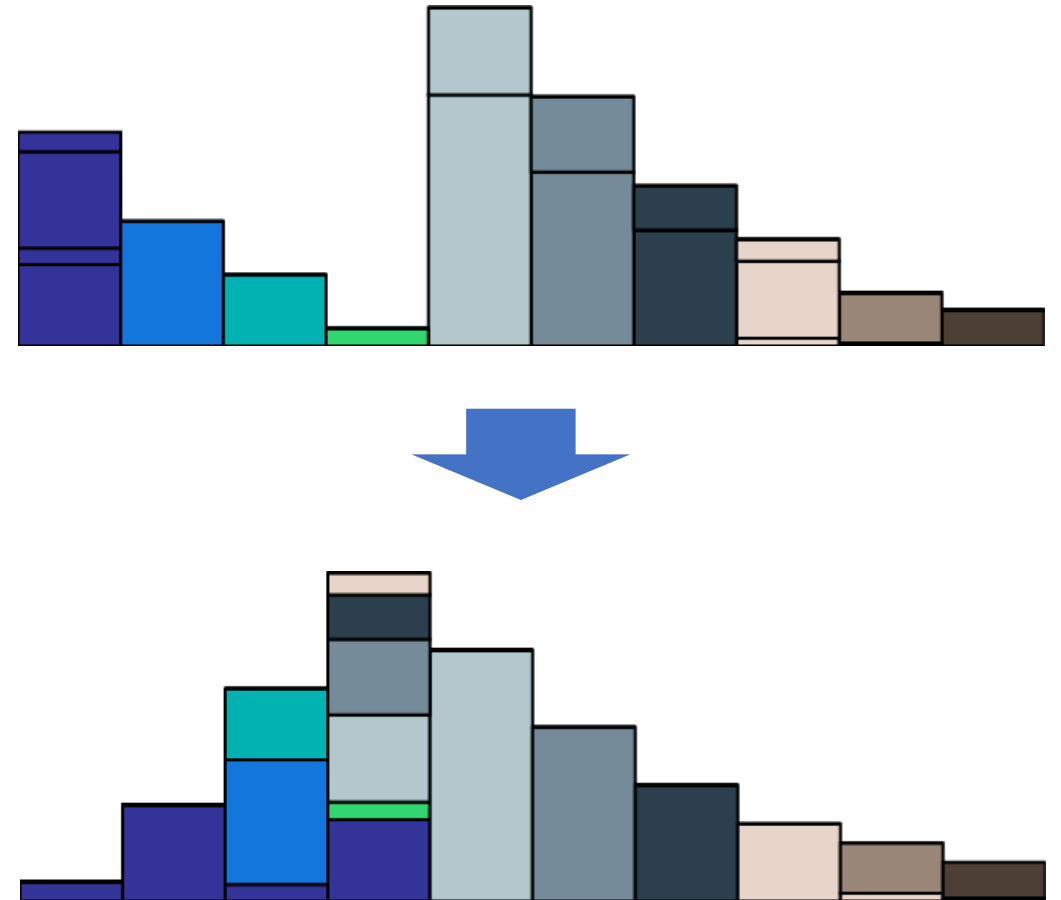
- μ and ν : Probability distributions
- $\Gamma(\mu, \nu)$: A set of possible joint distributions



p -Wasserstein Distance

A discrete setup:

The **minimum** amount of the **sum of mass $\times$ distance** to match one distribution to the other.



p -Wasserstein Distance

When the two distributions are normal distributions:

$$\mu = \mathcal{N}(m_1, C_1) \text{ and } \nu = \mathcal{N}(m_2, C_2),$$

2-Wasserstein distance becomes the following:

$$W(\mu, \nu) = \|m_1 - m_2\|_2^2 + \text{Tr} \left(C_1 + C_2 - 2 \left(C_2^{1/2} C_1 C_2^{1/2} \right)^{1/2} \right).$$

Fréchet Inception Distance (FID)

Compare the distributions of the real images and generated images in the latent space of **Inception v3 network.**

- Map all real and generated images to the latent space of the **Inception v3** network.
- Model each distribution as a **normal** distribution and compute the mean and covariance.
- Compute the 2-Wasserstein (Fréchet) distance.

Fréchet Inception Distance (FID)

Compare the distributions of the real images and generated images in the latent space of **Inception v3 network.**

- Map all real and generated images to the latent space of the **Inception v3** network.

Why?

- Model each distribution as a **normal** distribution and compute the mean and covariance.

Why?

- Compute the 2-Wasserstein (Fréchet) distance.

p -Wasserstein Distance

When the two distributions are normal distributions:

$$\mu = \mathcal{N}(m_1, C_1) \text{ and } \nu = \mathcal{N}(m_2, C_2),$$

2-Wasserstein distance becomes the following:

$$W(\mu, \nu) = \|m_1 - m_2\|_2^2 + \text{Tr} \left(C_1 + C_2 - 2 \left(C_2^{1/2} C_1 C_2^{1/2} \right)^{1/2} \right).$$

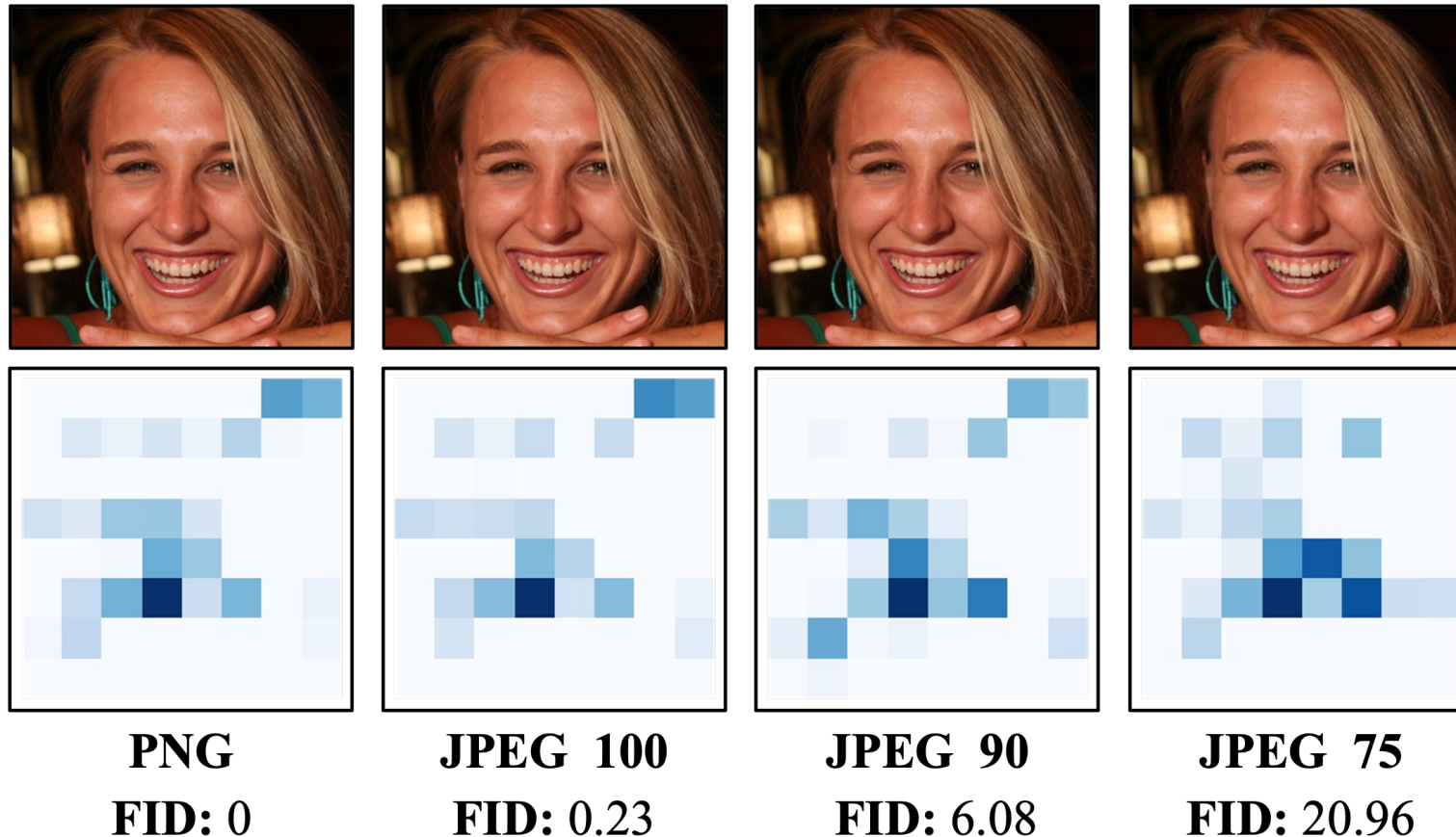
Fréchet Inception Distance (FID)

Compare the distributions of the real images and generated images in the latent space of **Inception v3 network.**

- Map all real and generated images to the latent space of the **Inception v3** network.
- Model each distribution as a **normal** distribution and compute the mean and covariance.
- Compute the 2-Wasserstein (Fréchet) distance.

Fréchet Inception Distance (FID)

Is FID a perfect metric for generative models?

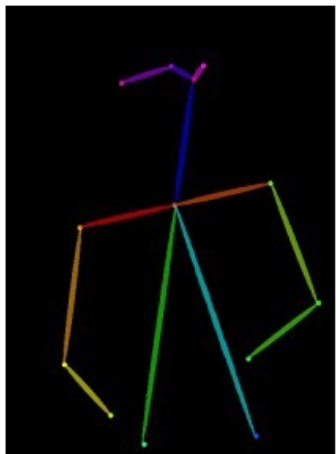


ControlNet: Few-Shot Adaptation

ControlNet



Input Canny edge



Input human pose



Default



“masterpiece of fairy tale, giant deer, golden antlers”



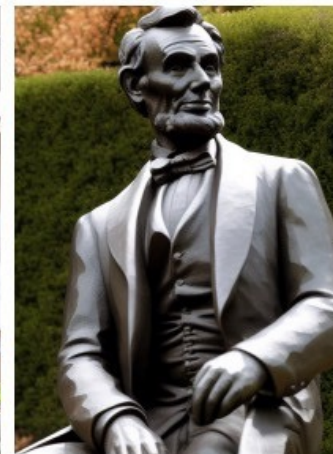
“..., quaint city Galic”



Default



“chef in kitchen”



“Lincoln statue”

ControlNet

Can we **convert** a pretrained unconditional image diffusion model into an **image-conditioned generative model** using a relatively much **smaller** set of input-output pairs ($\sim 100k$)?



ControlNet

Let's consider a special case where both the input condition and the output are **images**.

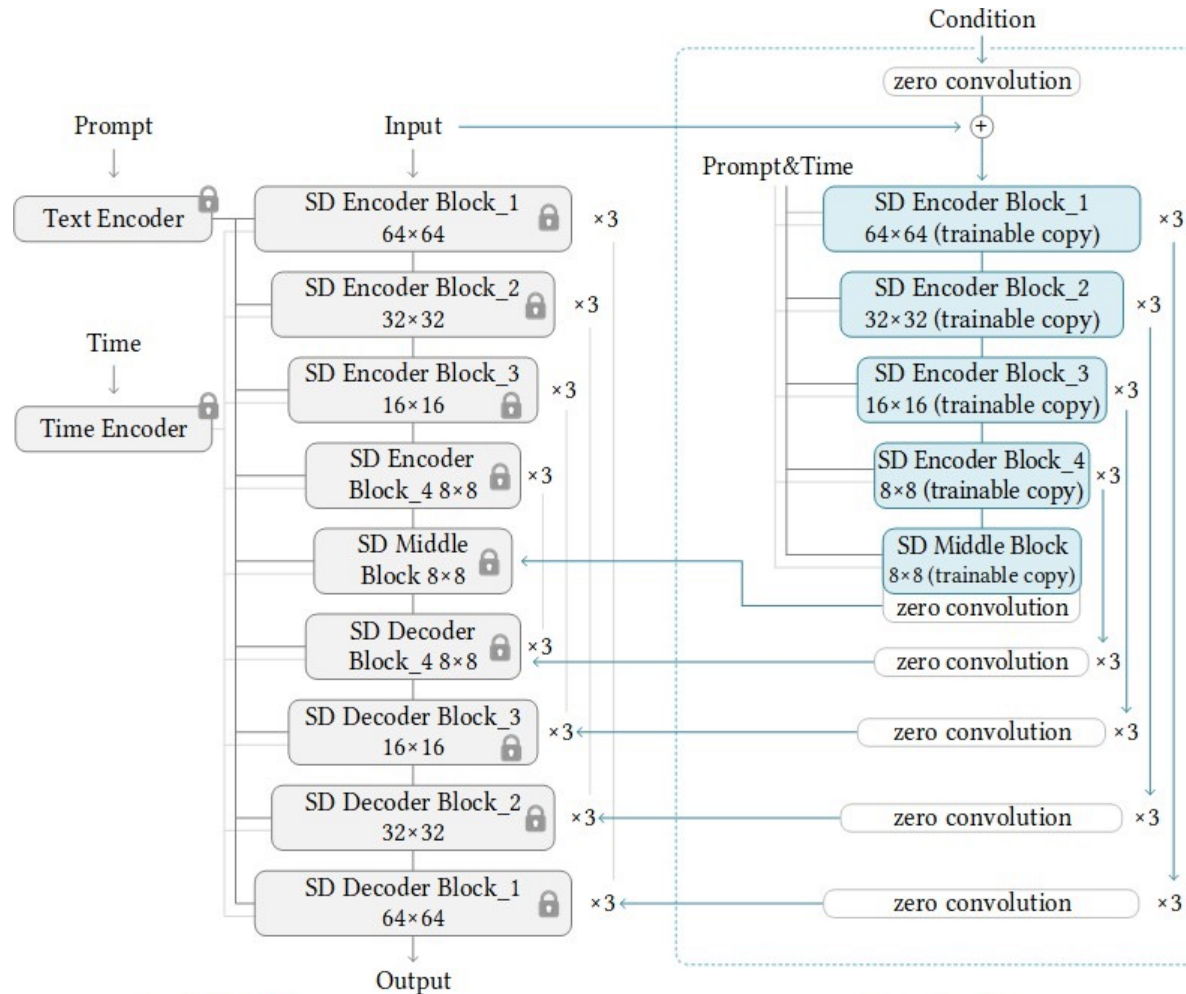


Fine-Tuning Approaches

Fine-tune the pretrained noise prediction network using only

- a small dataset, and
- a few additional learnable parameters, while
- keeping the network frozen!

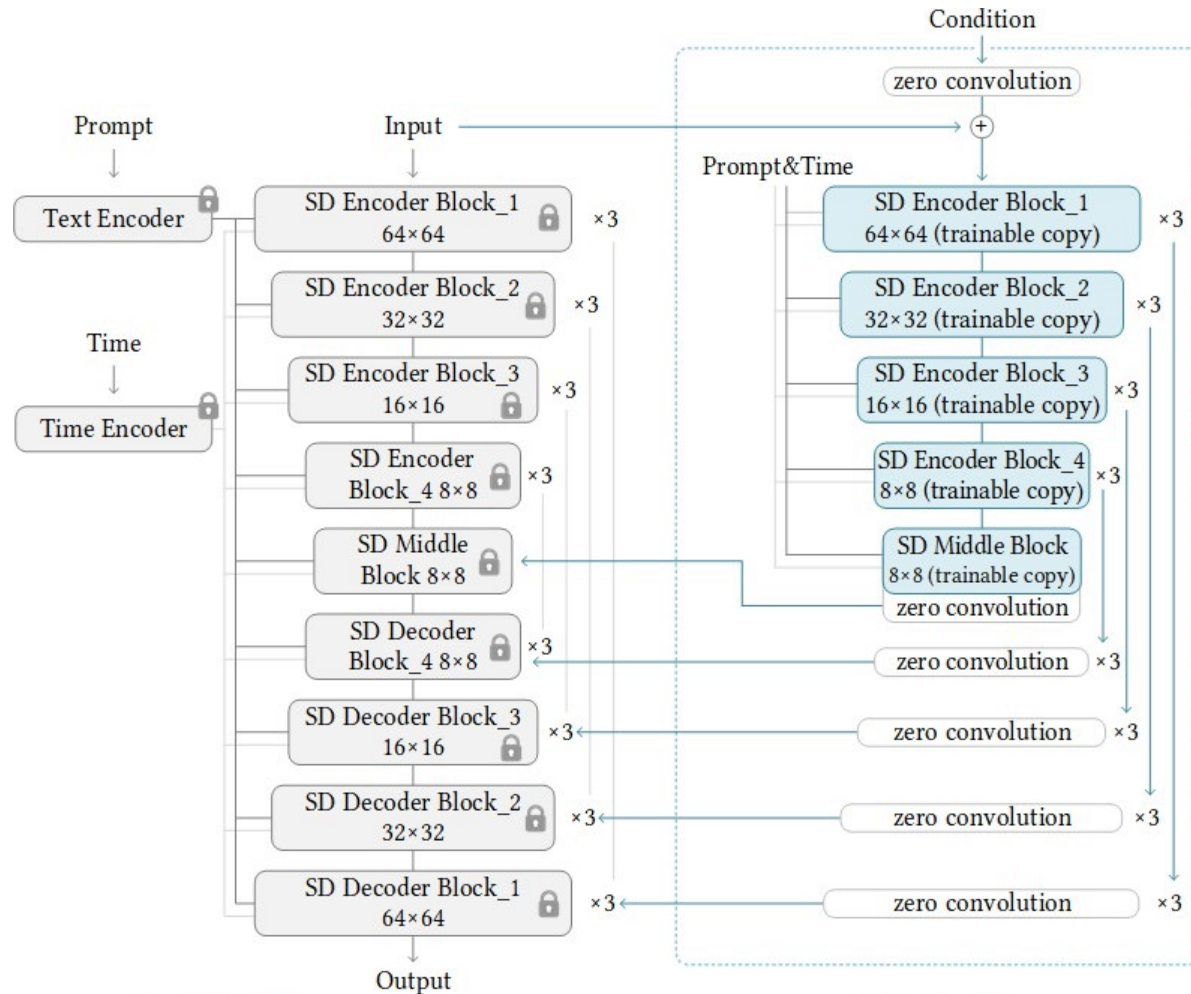
ControlNet



Key Idea:

Fully leverage the pretrained noise prediction network to process the **conditional** image.

ControlNet



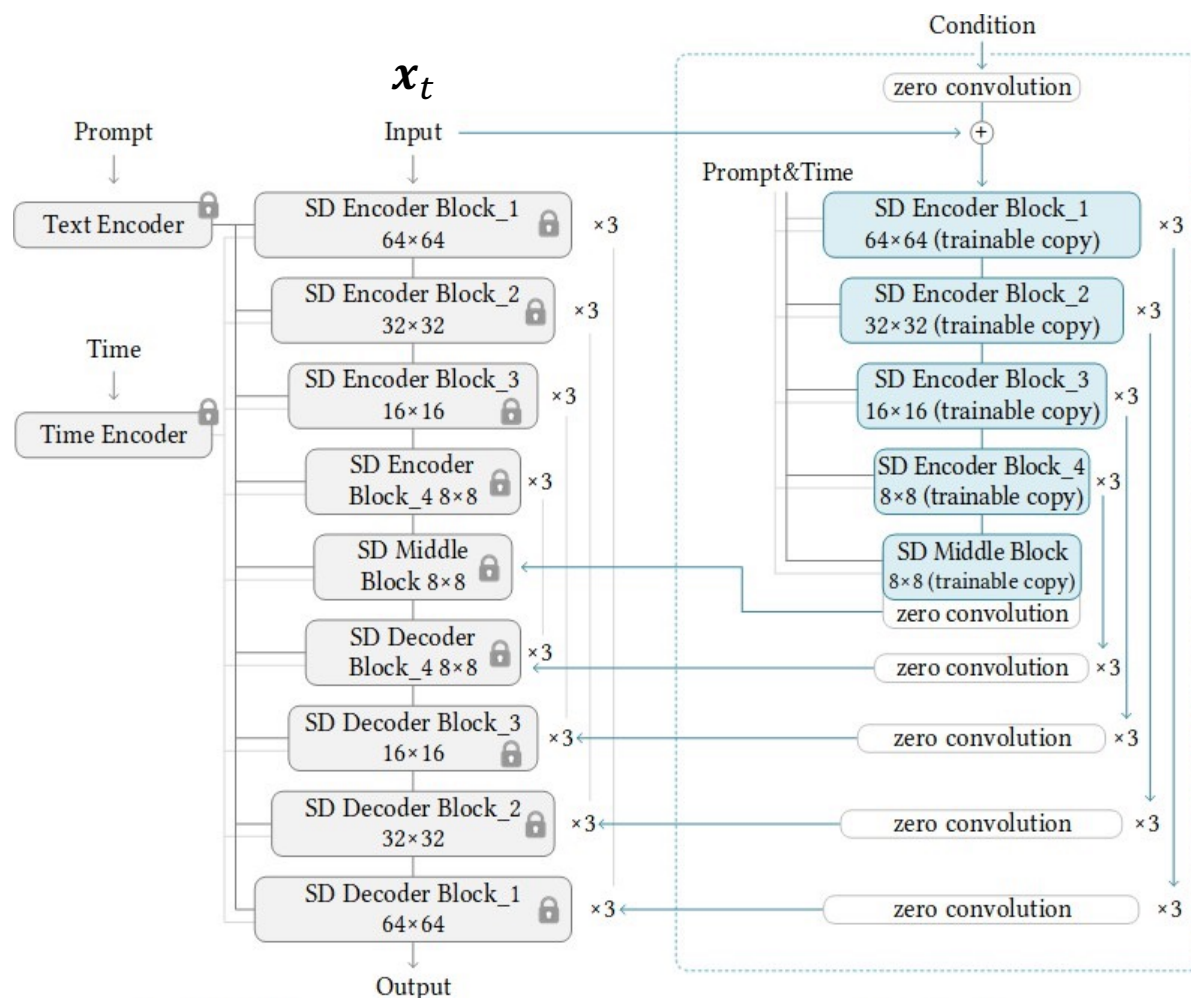
(a) Stable Diffusion

(b) ControlNet

Q.

What are the input and output of the noise prediction network?

ControlNet



(a) Stable Diffusion

$$\epsilon_{\theta}(x_t, t)$$

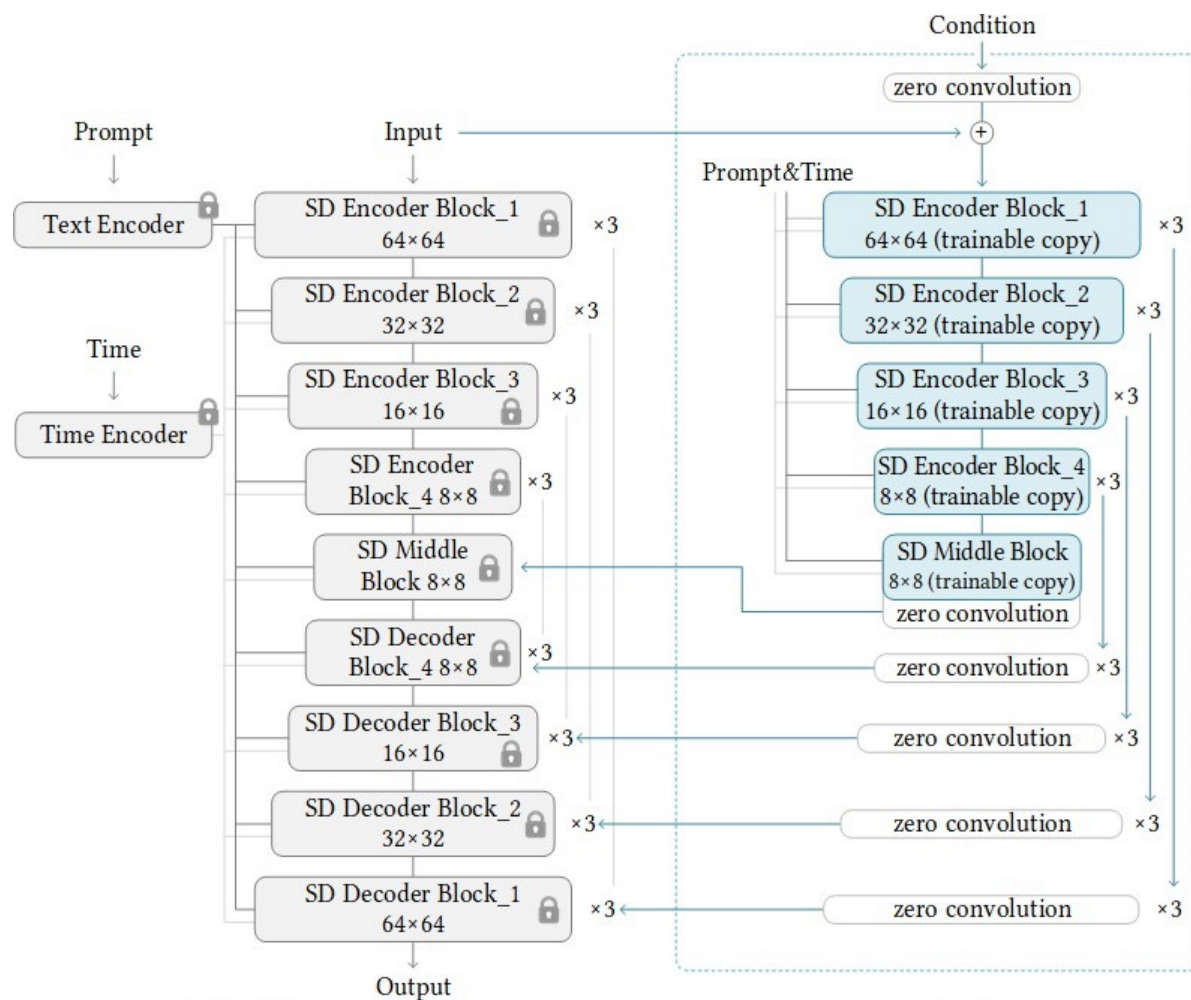
(b) ControlNet

A.

Input: latent/noisy image x_t

Output: Noise $\epsilon_{\theta}(x_t, t)$

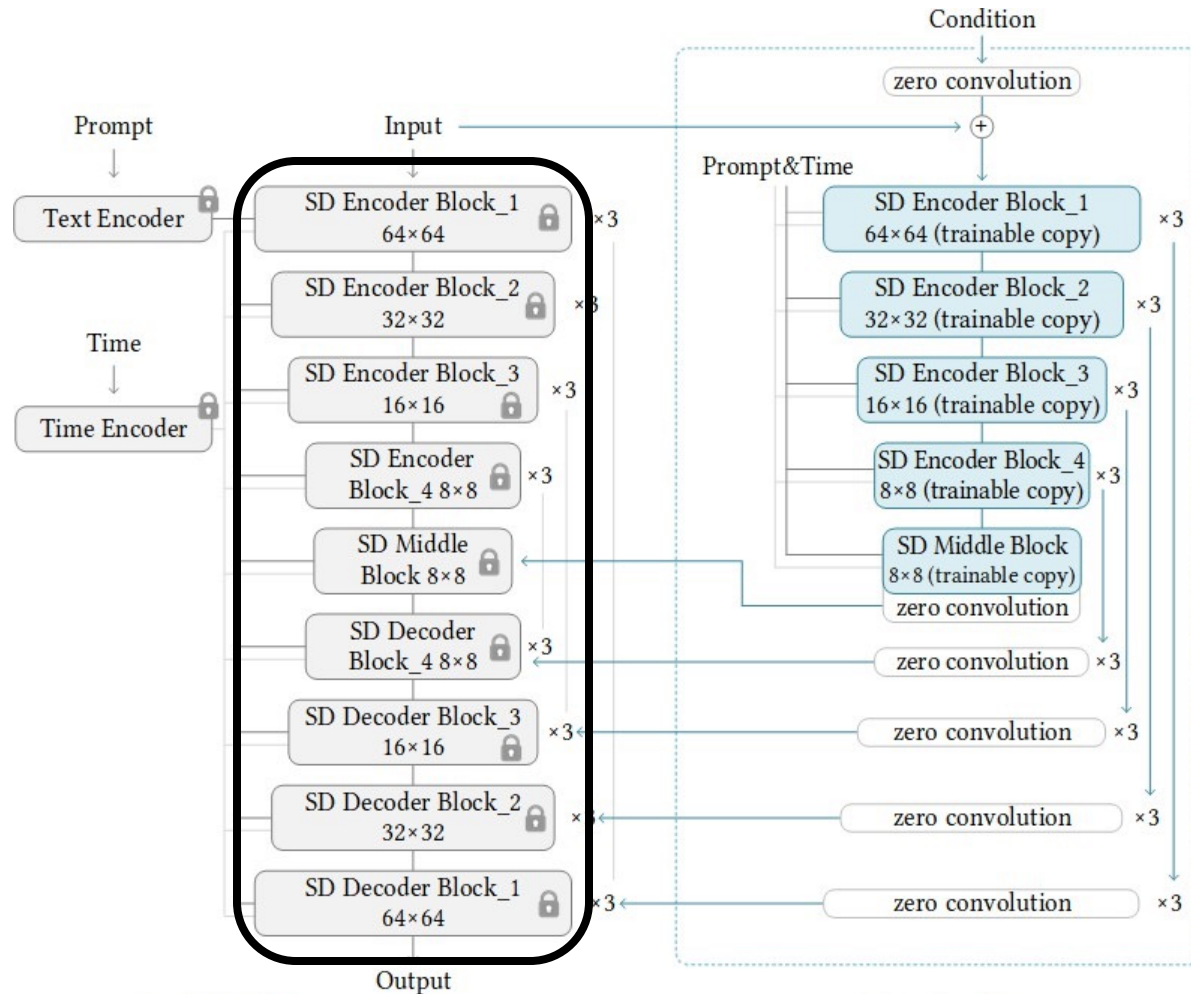
ControlNet



Key Idea:

Fully leverage the pretrained noise prediction network to process the **conditional** image.

ControlNet

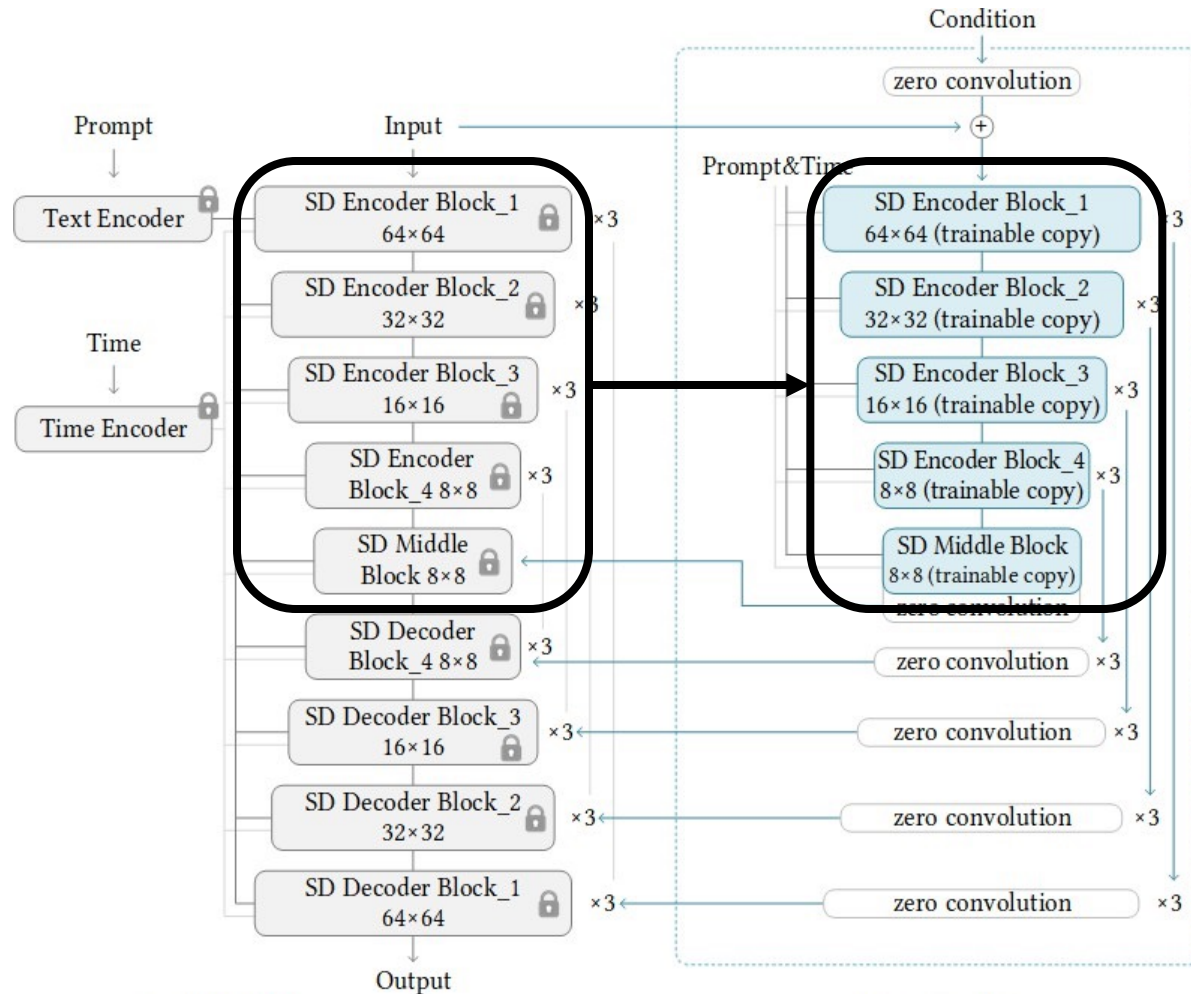


(a) Stable Diffusion

(b) ControlNet

1. Freeze the noise prediction network.

ControlNet

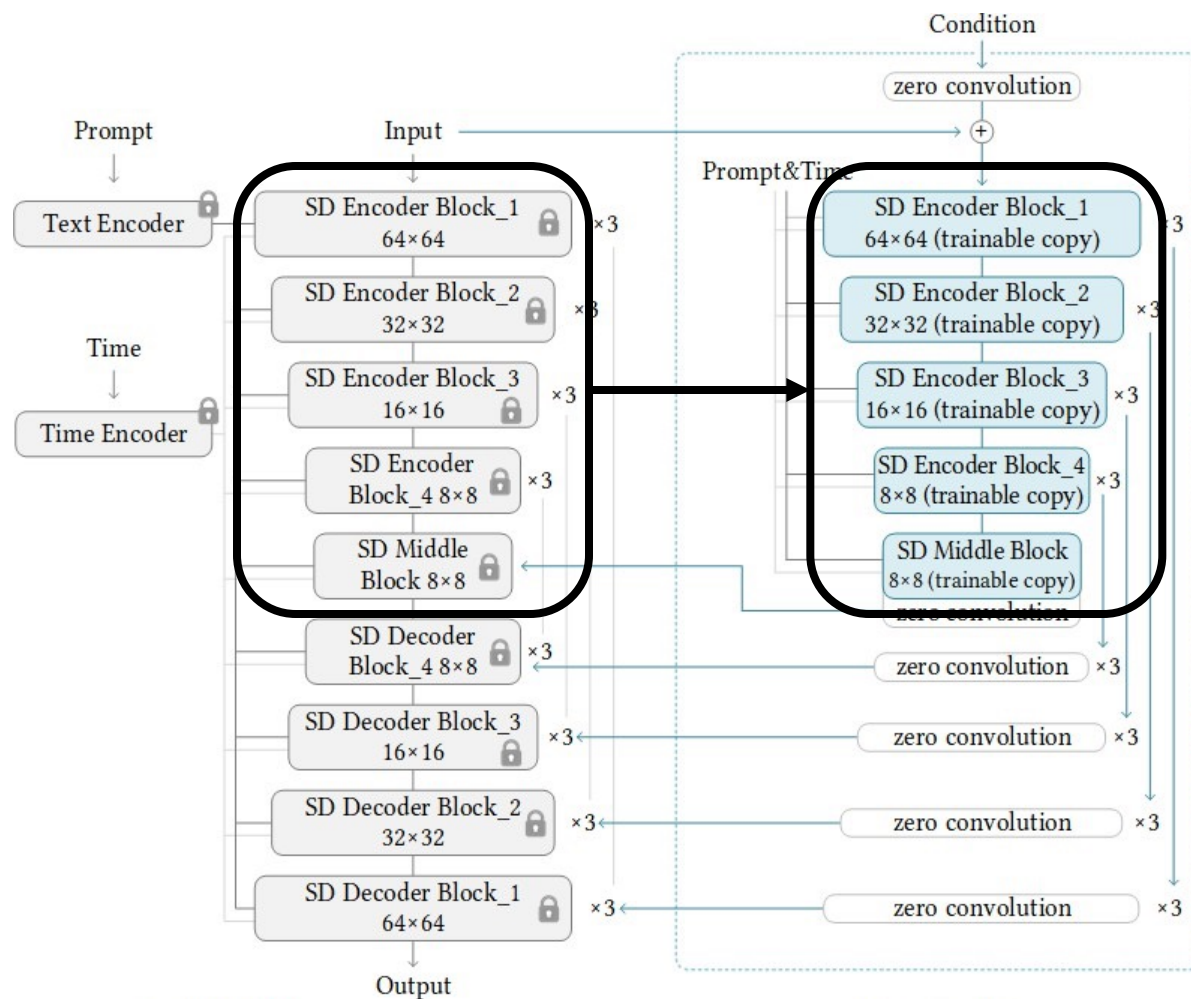


(a) Stable Diffusion

(b) ControlNet

2. For the encoding of the **conditional** image, **copy the pretrained encoder parameters** while allowing them to be updated during fine-tuning.

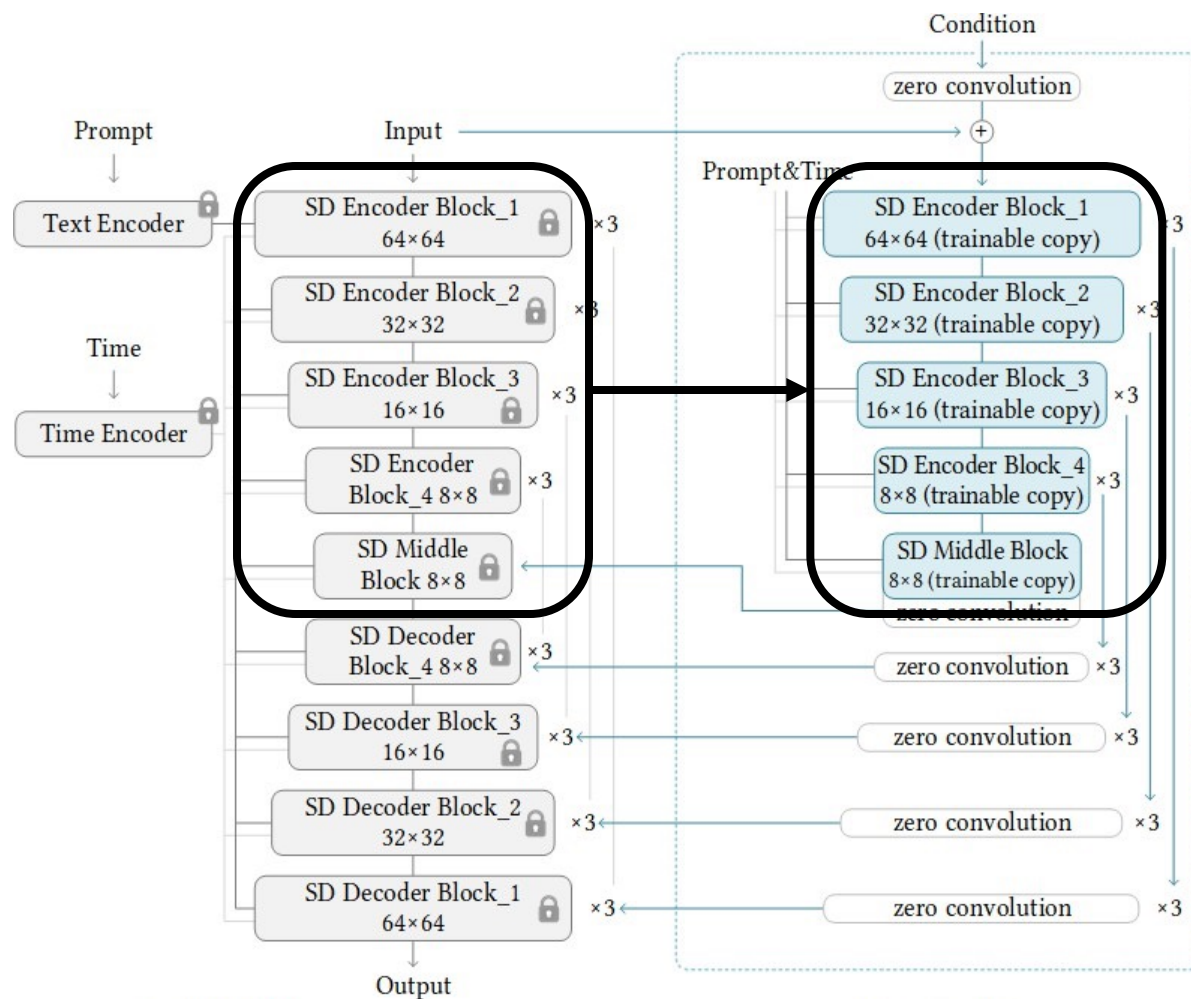
ControlNet



Q.

為什麼要 copy 架構跟參數而不是從頭訓練？

ControlNet



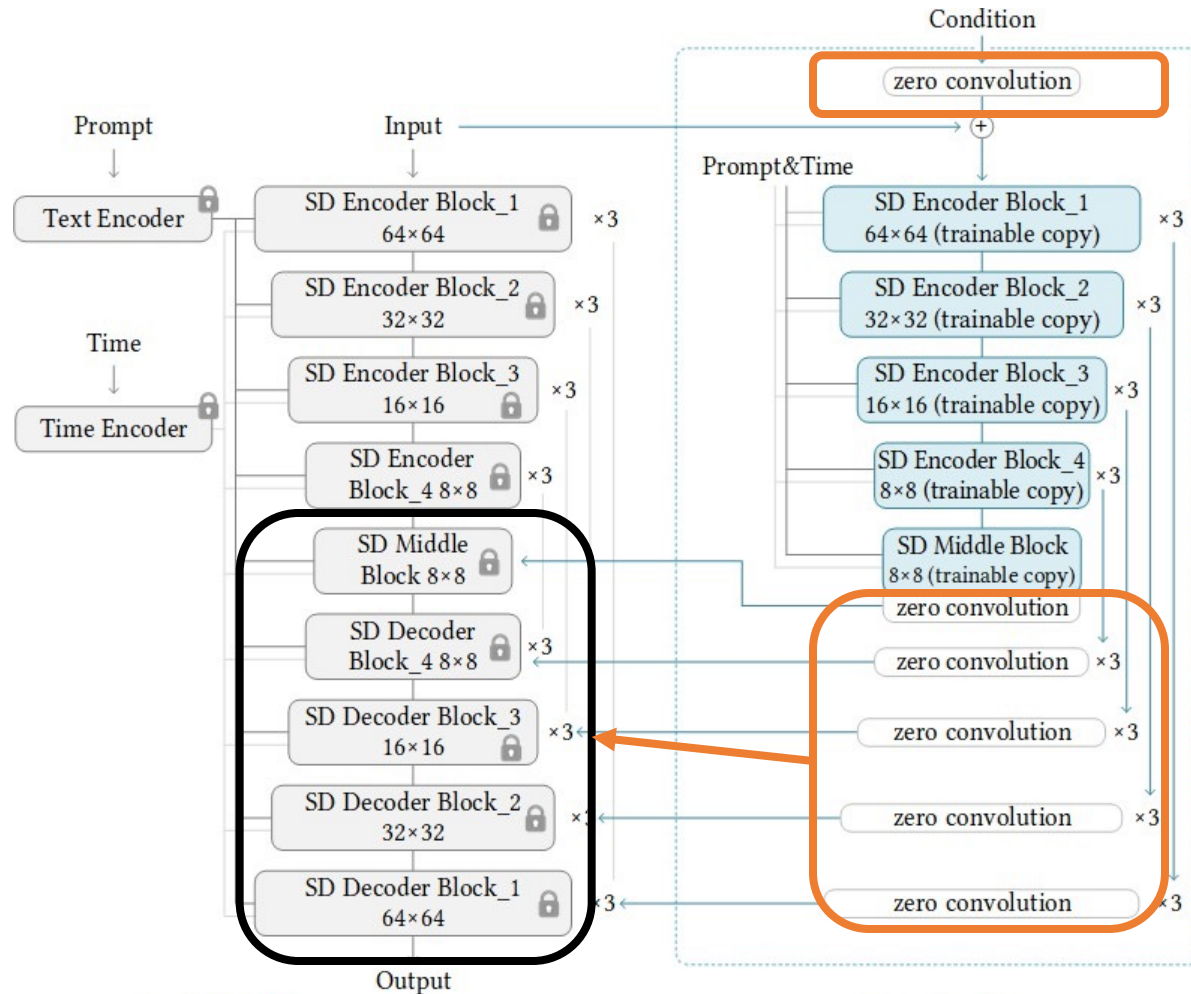
(a) Stable Diffusion

(b) ControlNet

A.

保留 pre-trained model 的 feature extraction 能力，原本 pre-trained Stable Diffusion encoder 已經學會了理解影像的語意、extract 不同層級的 feature，從頭訓練會損失這些寶貴的 prior

ControlNet



3. Combine the encoded conditional image information with the noisy image using **zero convolution**.

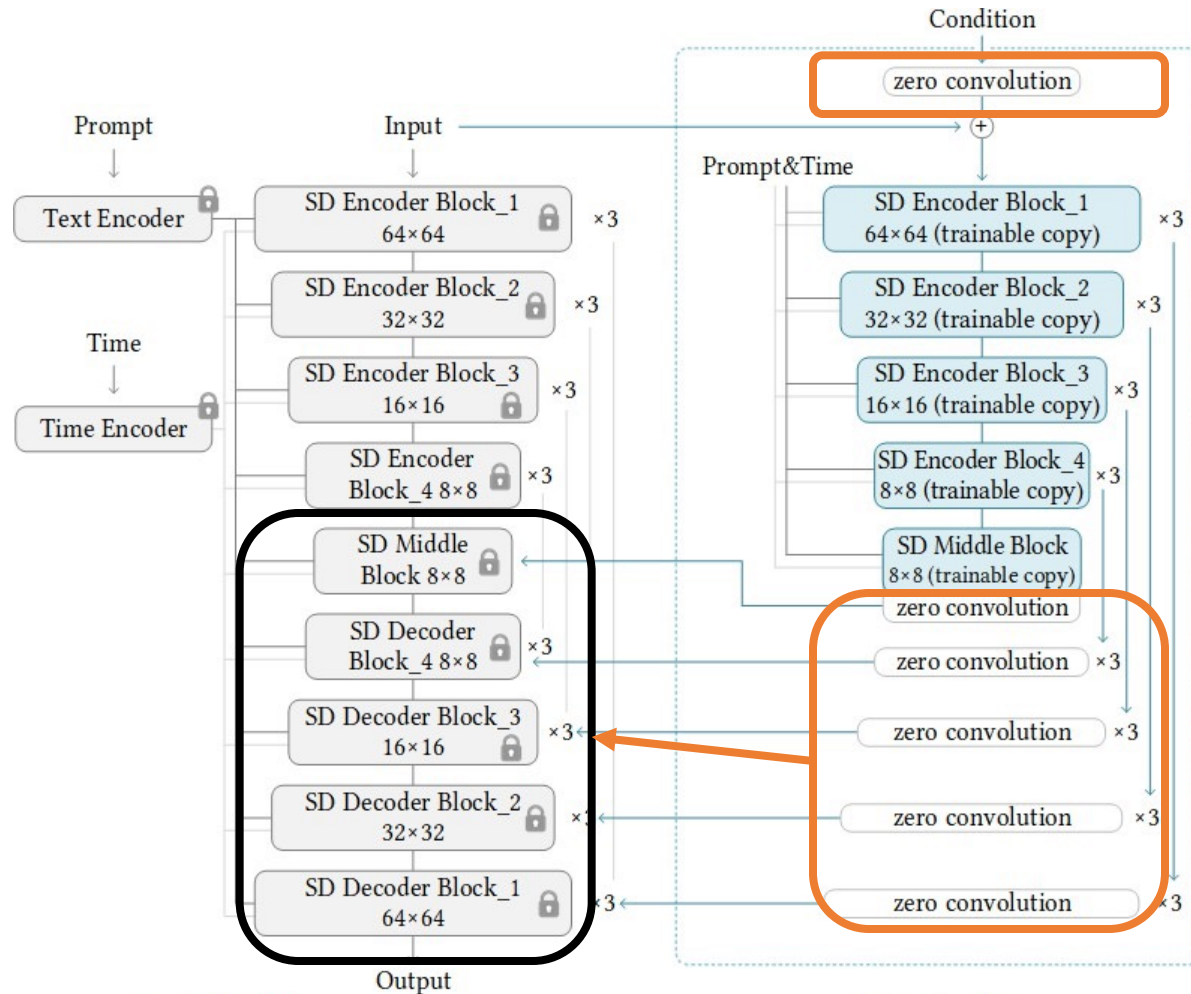
(b) ControlNet

ControlNet

Zero Convolution Z is a 1×1 convolution layer with learnable weight (scaling) parameters $\mathbf{a}$ and bias (offset) parameters $\mathbf{b}$, both of which are initialized with zero:

$$Z(\mathbf{x}; \mathbf{a}, \mathbf{b}) = \mathbf{a}\mathbf{x} + \mathbf{b}$$

ControlNet



(a) Stable Diffusion

(b) ControlNet

3. Combine the encoded conditional image information with the noisy image using **zero convolution**.

ControlNet

Zero Convolution Z is a 1×1 convolution layer with learnable weight (scaling) parameters $\mathbf{a}$ and bias (offset) parameters $\mathbf{b}$, both of which are initialized with zero:

$$Z(\mathbf{x}; \mathbf{a}, \mathbf{b}) = \mathbf{a}\mathbf{x} + \mathbf{b}$$

ControlNet

Zero Convolution Z is a 1×1 convolution layer with learnable weight (scaling) parameters $\mathbf{a}$ and bias (offset) parameters $\mathbf{b}$, both of which are **initialized with zero**:

$$Z(\mathbf{x}; \mathbf{a}, \mathbf{b}) = \mathbf{a}\mathbf{x} + \mathbf{b}$$

Q. $\mathbf{a}$ 跟 $\mathbf{b}$ 的 dimension 是什麼？假設 $\mathbf{x}$ 的 dimension 為 $[H, W, F]$

ControlNet

Zero Convolution Z is a 1×1 convolution layer with learnable weight (scaling) parameters $\mathbf{a}$ and bias (offset) parameters $\mathbf{b}$, both of which are initialized with zero:

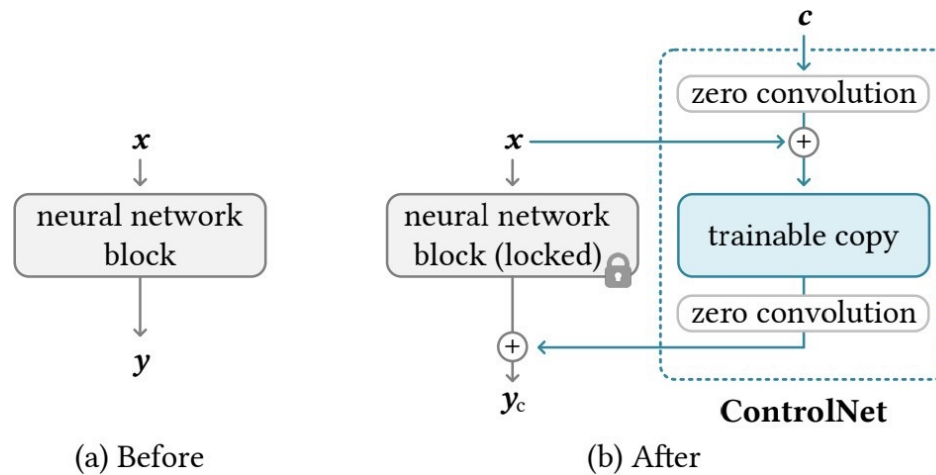
$$Z(\mathbf{x}; \mathbf{a}, \mathbf{b}) = \mathbf{a}\mathbf{x} + \mathbf{b}$$

A. $\mathbf{a}$ 跟 $\mathbf{b}$ 的 dimension 都是 $[F]$

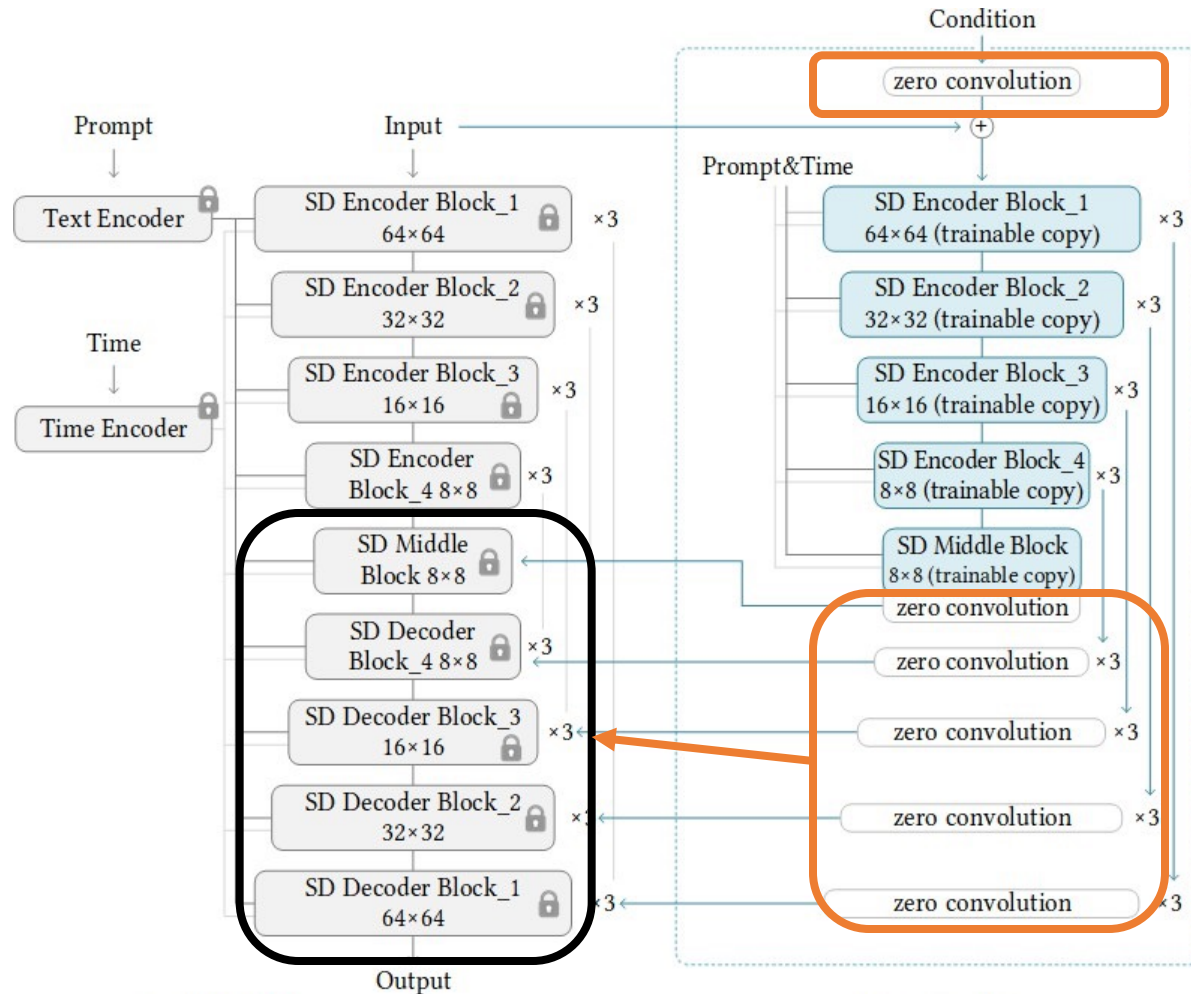
ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$



ControlNet

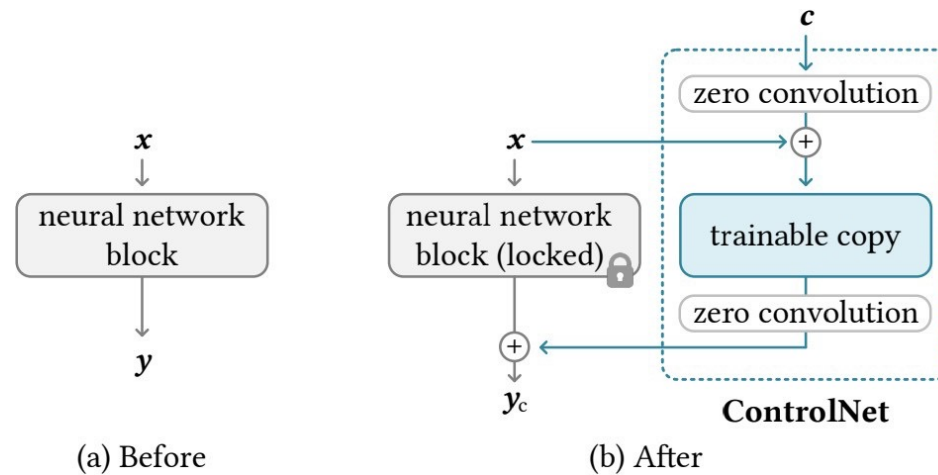


3. Combine the encoded conditional image information with the noisy image using **zero convolution**.

ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

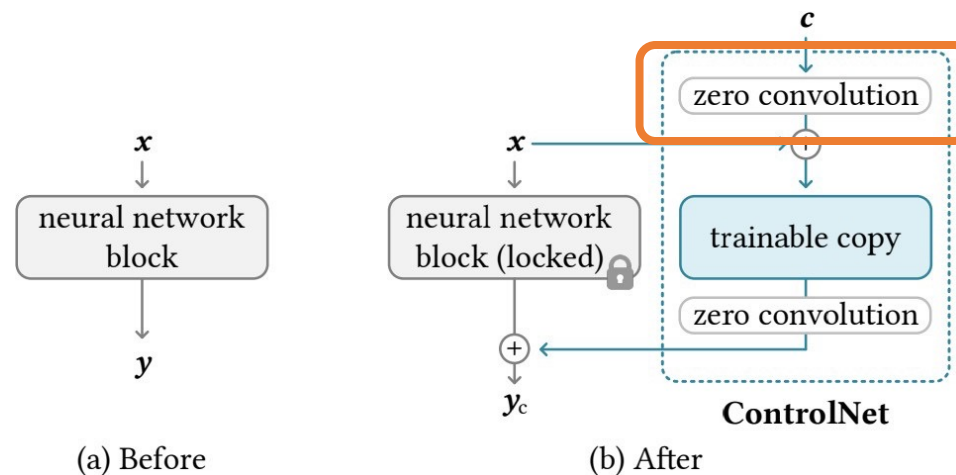
$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$



ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

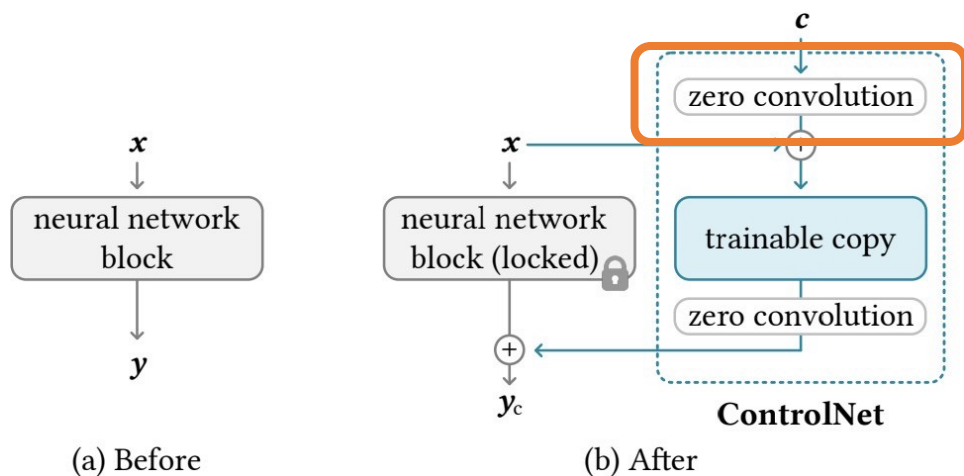


Q. Why do we need this zero convolution?

ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

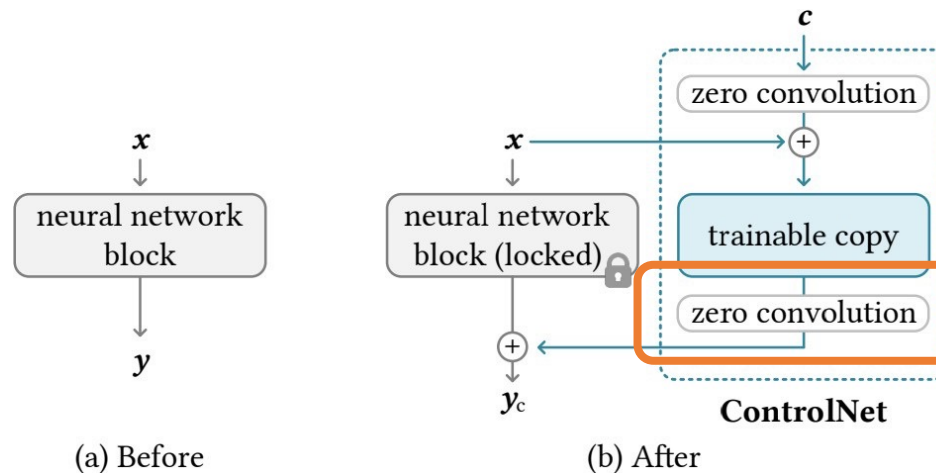


A. 處理 condition
Zero convolution 可以確保開始時 trainable copy 只看原始輸入 x ，並逐漸學習如何把 condition c 融合進來

ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

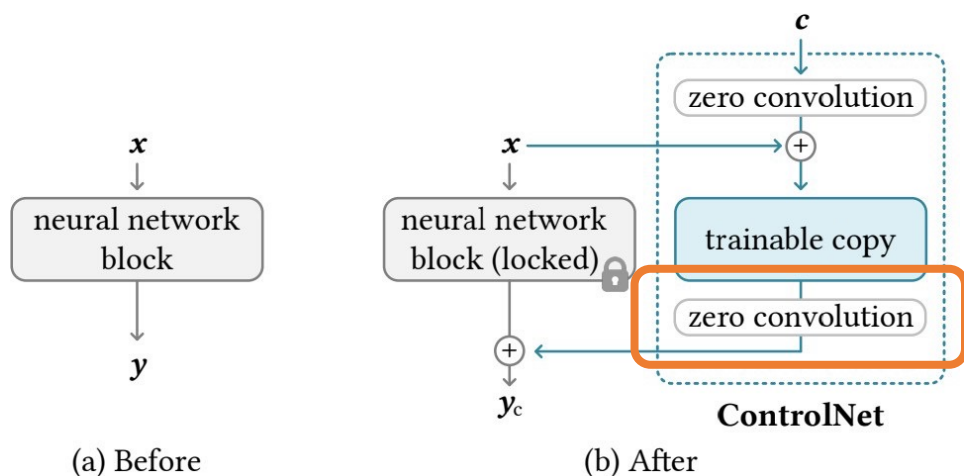


Q. Why do we need this zero convolution?

ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$\mathbf{y}_c = F(\mathbf{x}; \Theta) + Z(F(\mathbf{x} + Z(\mathbf{c}; a_1, b_1); \Theta_c); a_2, b_2)$$

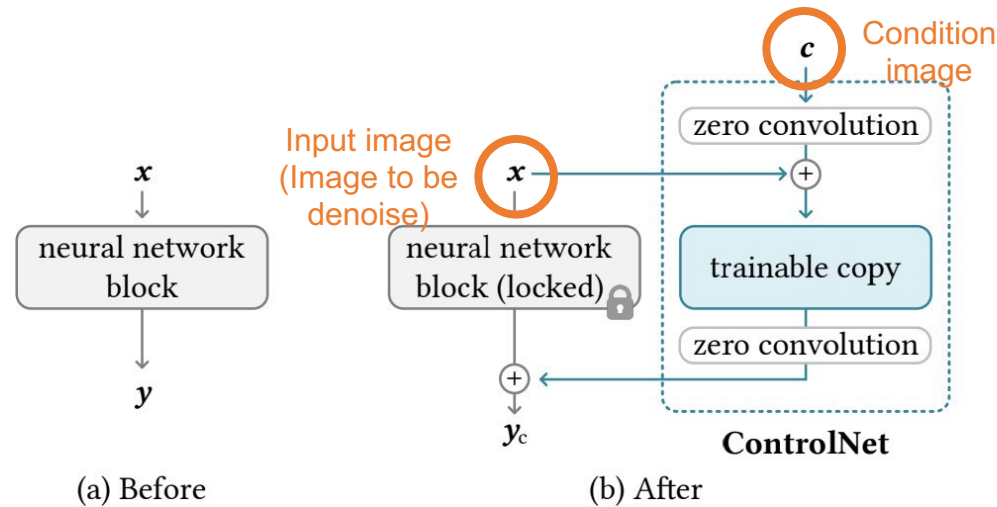


A. 讓 condition 影響輸出
Zero convolution 確保開始時完全不影響原始 network output $\mathbf{y}_c$ ，並逐漸學習適當的控制強度

ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

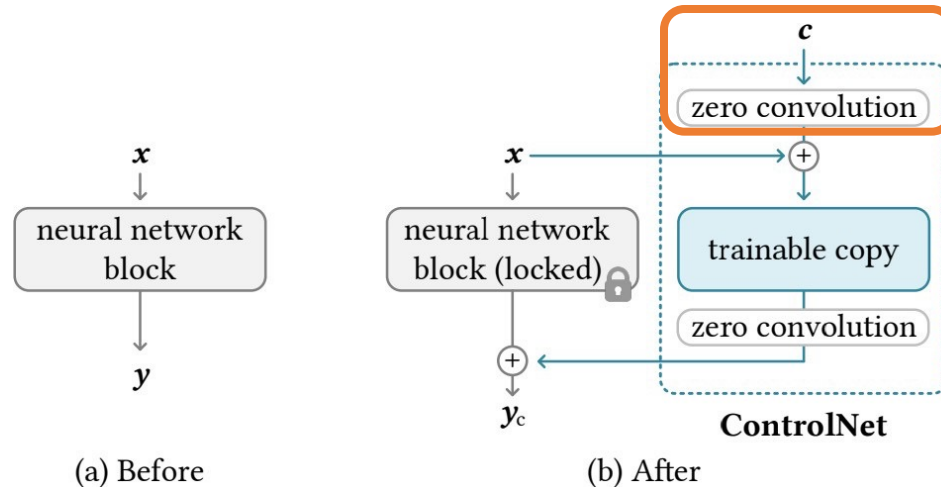


ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1) \Theta_c); a_2, b_2)$$

Zero in the beginning since $a_1 = b_1 = 0$.

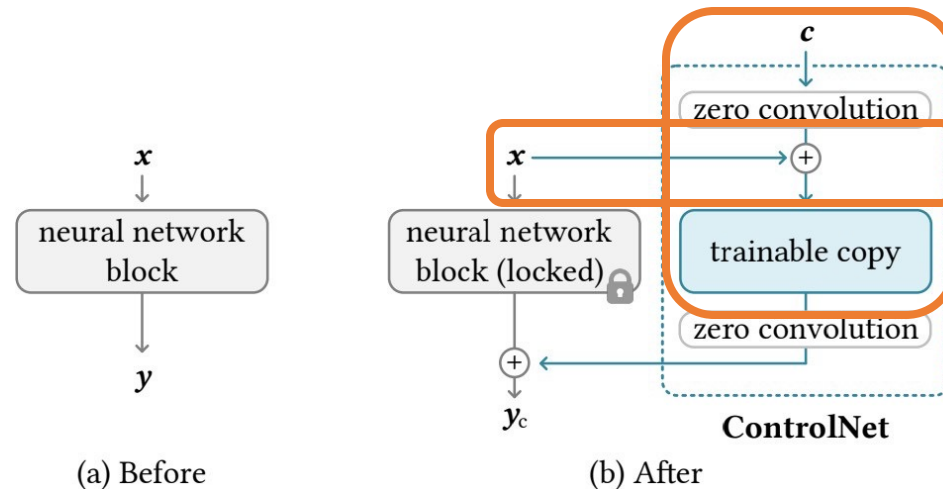


ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

Same as $F(x; \Theta)$ in the beginning since $\Theta_c = \Theta$.

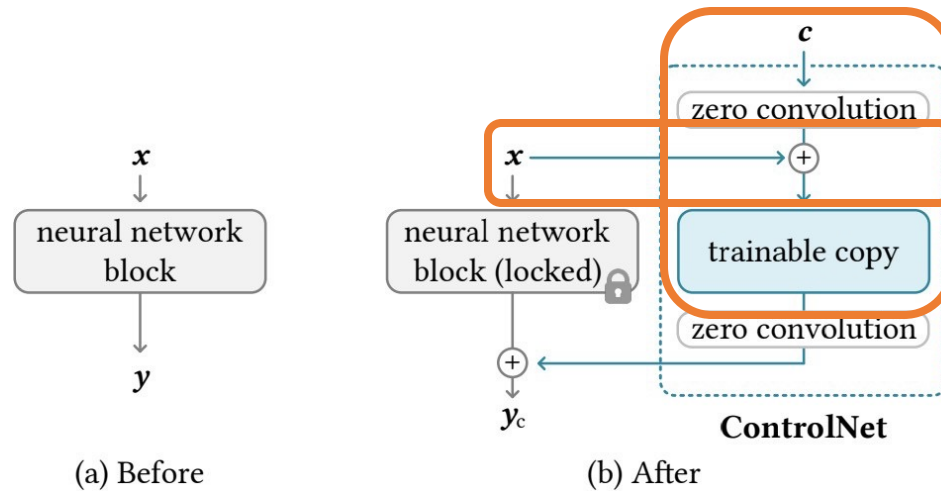


ControlNet

For encoding of the conditional image c , **begin with the state where the input is x** , while gradually incorporating c .

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

Same as $F(x; \Theta)$ in the beginning since $\Theta_c = \Theta$.

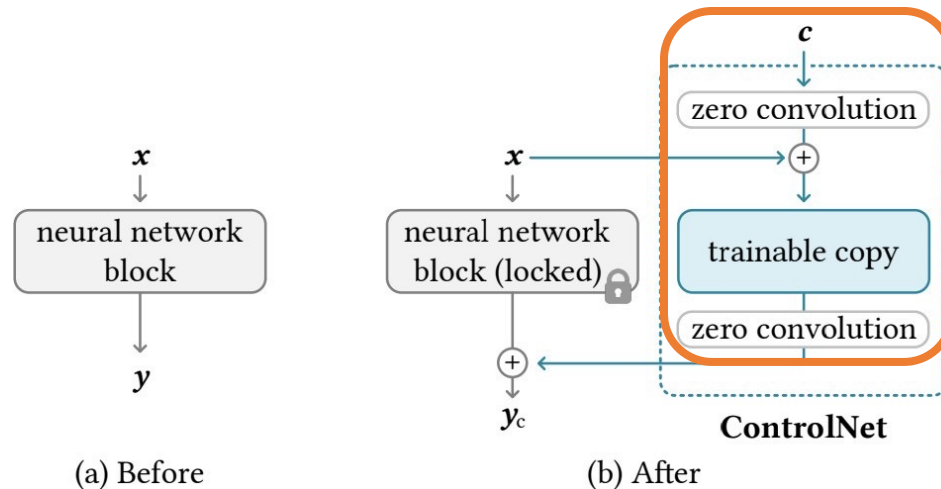


ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

Zero in the beginning since $a_1 = b_1 = 0$.

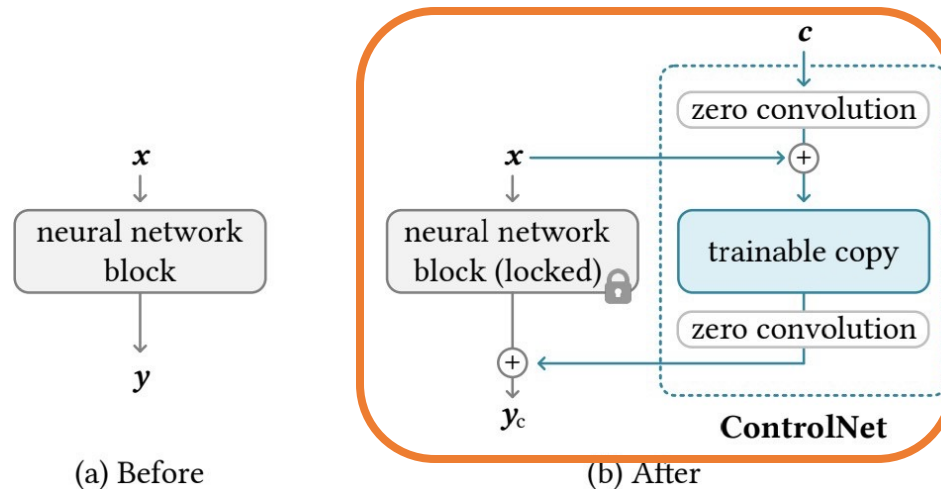


ControlNet

ControlNet modifies each neural network block using zero convolution as follows:

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

Same as $F(x; \Theta)$ in the beginning.

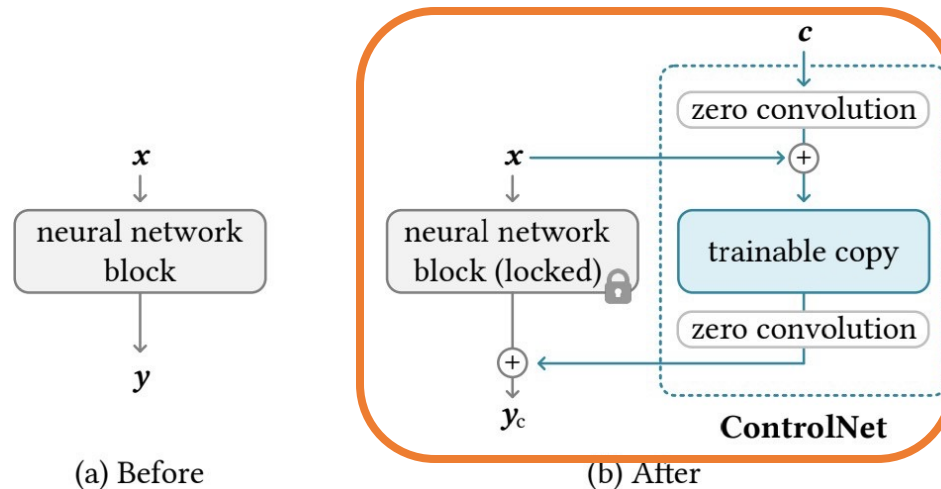


ControlNet

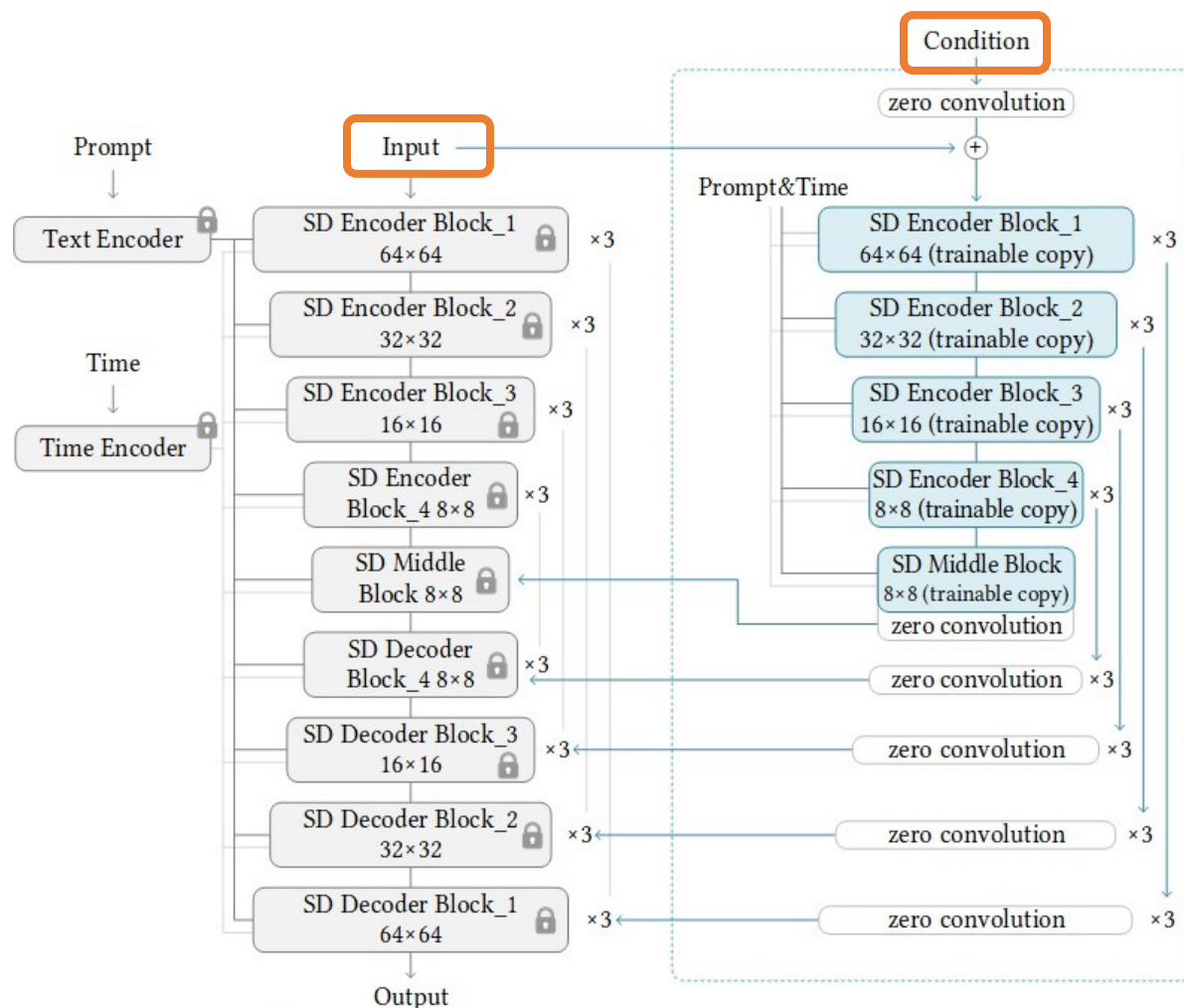
Also for the noise prediction, gradually incorporate the output of conditional image encoding.

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

Same as $F(x; \Theta)$ in the beginning.



ControlNet



(a) Stable Diffusion

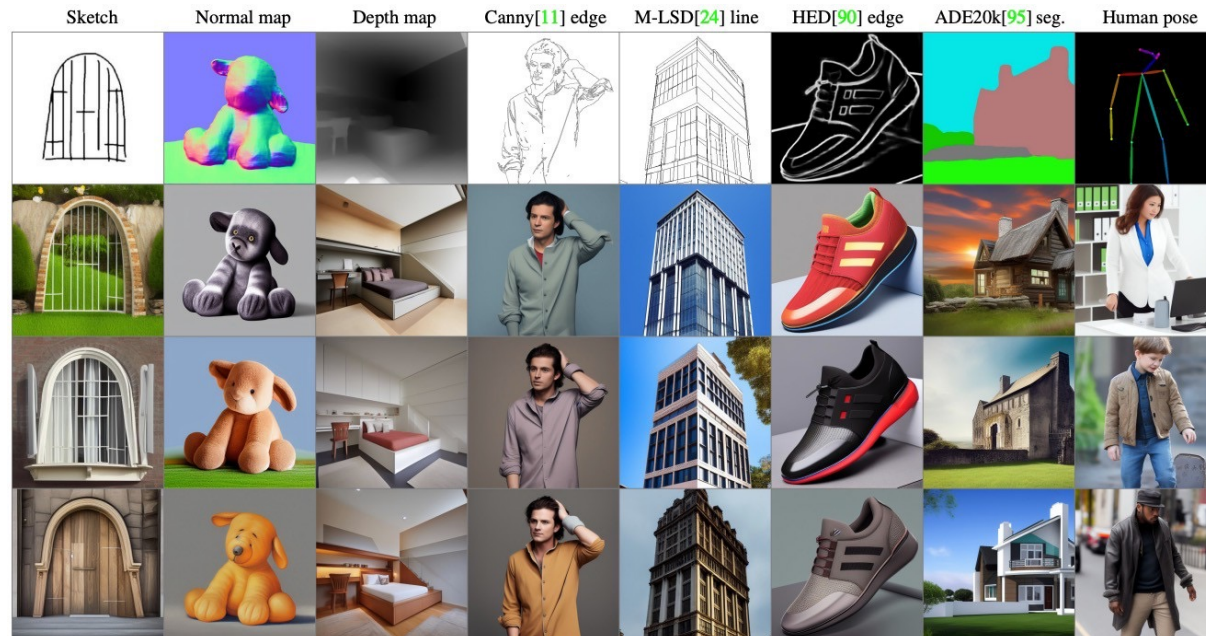
(b) ControlNet

需注意 condition c 的資料型態與 dimension 需要與 input 完全相同，例如：同樣都是 $512 \times 512 \times 3$ 的影像

且在 latent/Stable Diffusion 的情況下，condition c 同樣也需要先經過 encoder 變成 latent

ControlNet

- The idea can be used for any image conditioning.
- The training dataset is “~100k in size, which is 50,000 time smaller than the LAION-5B.”

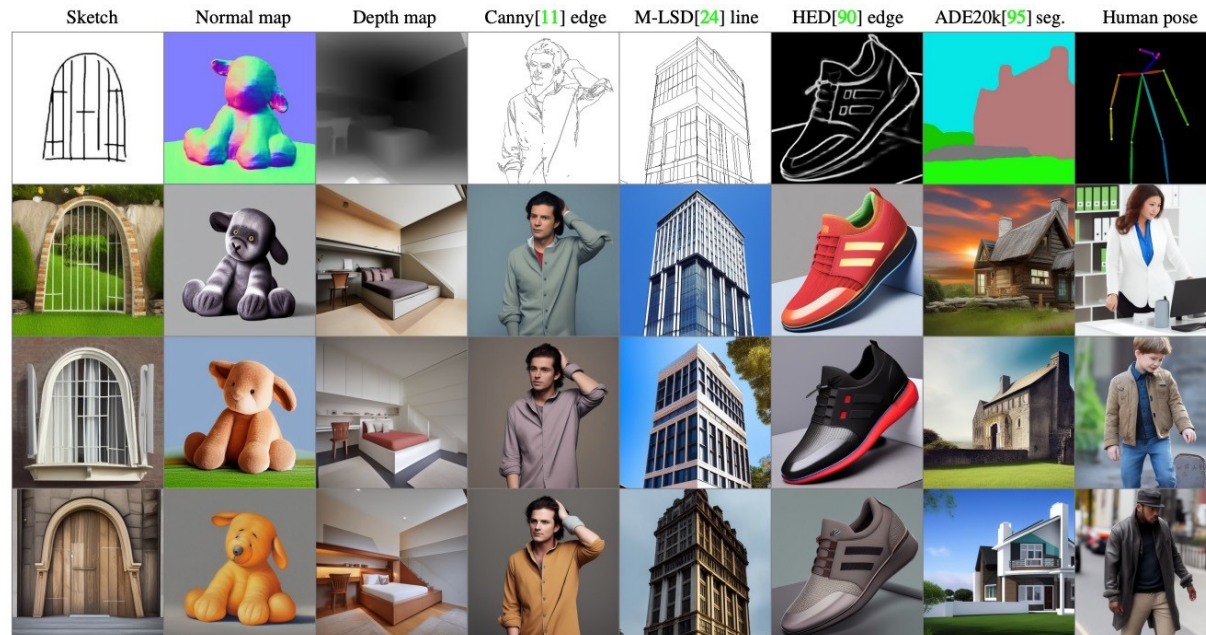


ControlNet

- <https://github.com/lllyasviel/ControlNet>

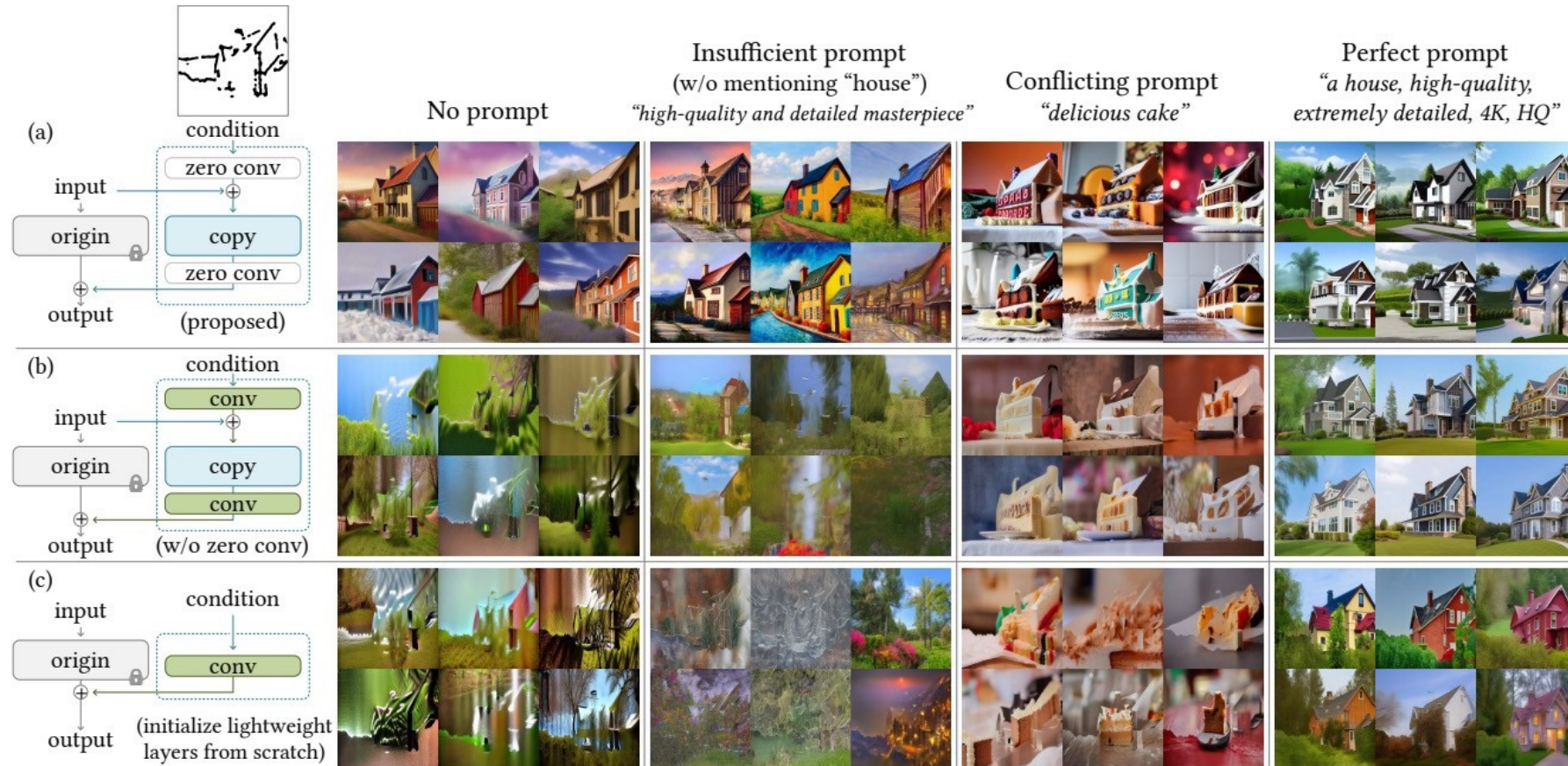
ControlNet

- The idea can be used for any image conditioning.
- The training dataset is “~100k in size, which is 50,000 time smaller than the LAION-5B.”



ControlNet

Ablative study of different architectures.



Other Method: GLIGEN



Caption: "a photo of a hybrid between a bee and a rabbit"
Grounded text: **hybrid between a bee and a rabbit**, flower



Caption: "Pikachu is under a tree, digital art"
Grounded text: **Pikachu**, tree; Grounded style image: blue inset



Caption: "A dog / bird / helmet / backpack is on the grass"
Grounded image: red inset



Caption: "superman / monkey / Horner Simpson / is scratching its head"
Grounded keypoints: plotted dots on the left image



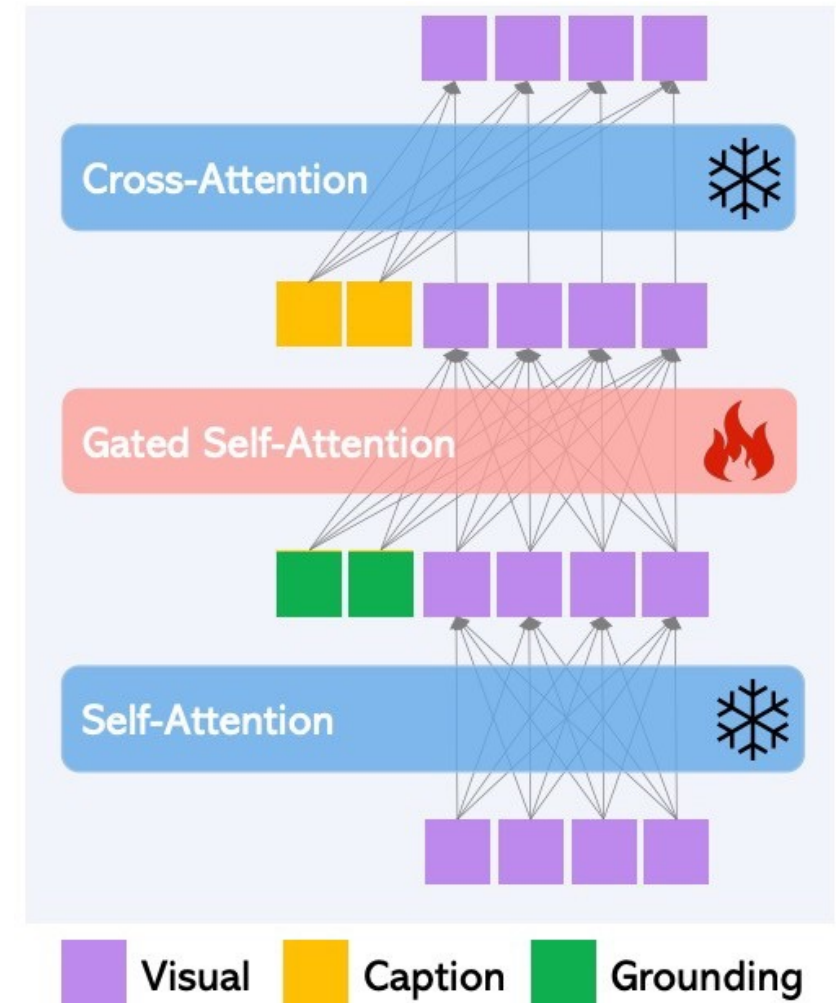
Caption: "A vibrant colorful bird sitting on tree branch"
Grounded depth map: the left image



Caption: "The beautiful scenery of a clam village near the sea"
Grounded HED map: the left image

Other Method: GLIGEN

Fine-tune the additional gated self-attention modules while freezing the network parameters.



Other Method: GLIGEN



Caption: "a photo of a hybrid between a bee and a rabbit"
Grounded text: **hybrid between a bee and a rabbit**, flower



Caption: "Pikachu is under a tree, digital art"
Grounded text: **Pikachu**, tree; Grounded style image: blue inset



Caption: "A dog / bird / helmet / backpack is on the grass"
Grounded image: red inset



Caption: "superman / monkey / Horner Simpson / is scratching its head"
Grounded keypoints: plotted dots on the left image



Caption: "A vibrant colorful bird sitting on tree branch"
Grounded depth map: the left image



Caption: "The beautiful scenery of a clam village near the sea"
Grounded HED map: the left image

ControlNet

Zero Convolution Z is a 1×1 convolution layer with learnable weight (scaling) parameters $\mathbf{a}$ and bias (offset) parameters $\mathbf{b}$, both of which are initialized with zero:

$$Z(\mathbf{x}; \mathbf{a}, \mathbf{b}) = \mathbf{a}\mathbf{x} + \mathbf{b}$$

Personalization

Personalization



LoRA

Personalization

Input images



A [V] backpack in the Grand Canyon



A [V] backpack with the night sky



A [V] backpack in the city of Versailles



A wet [V] backpack in water



A [V] backpack in Boston

DreamBooth

Personalization

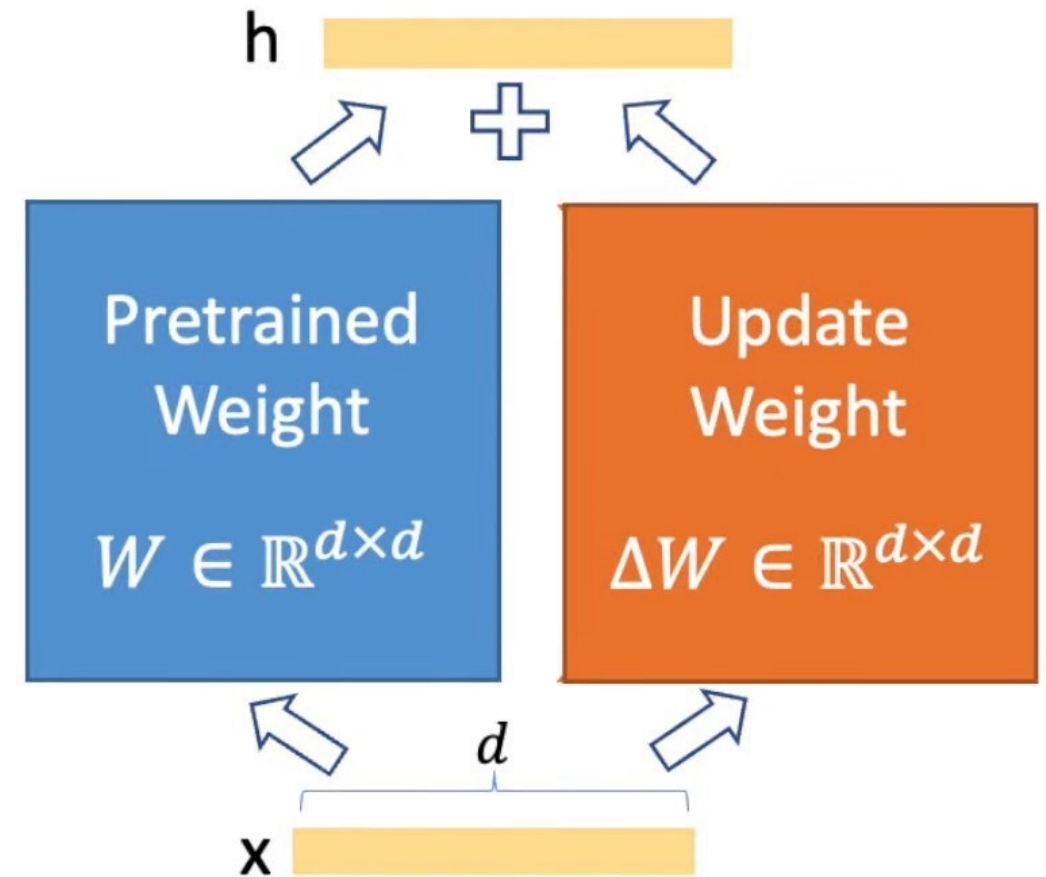
I want to generate images of myself or my objects!

How? Use the **fine-tuning** approach.

Introduce a very few learnable parameters.

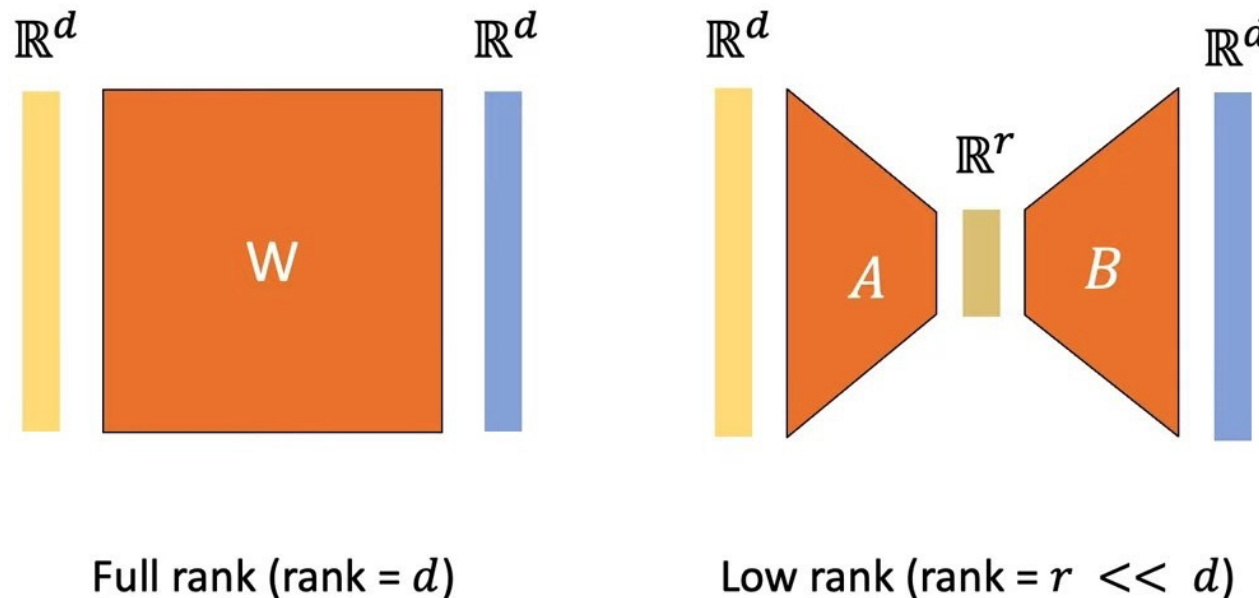
Low-Rank Adaptation

Similar to ControlNet, create an **additional branch** for each layer that takes the same input and outputs offsets.



Low-Rank Adaptation

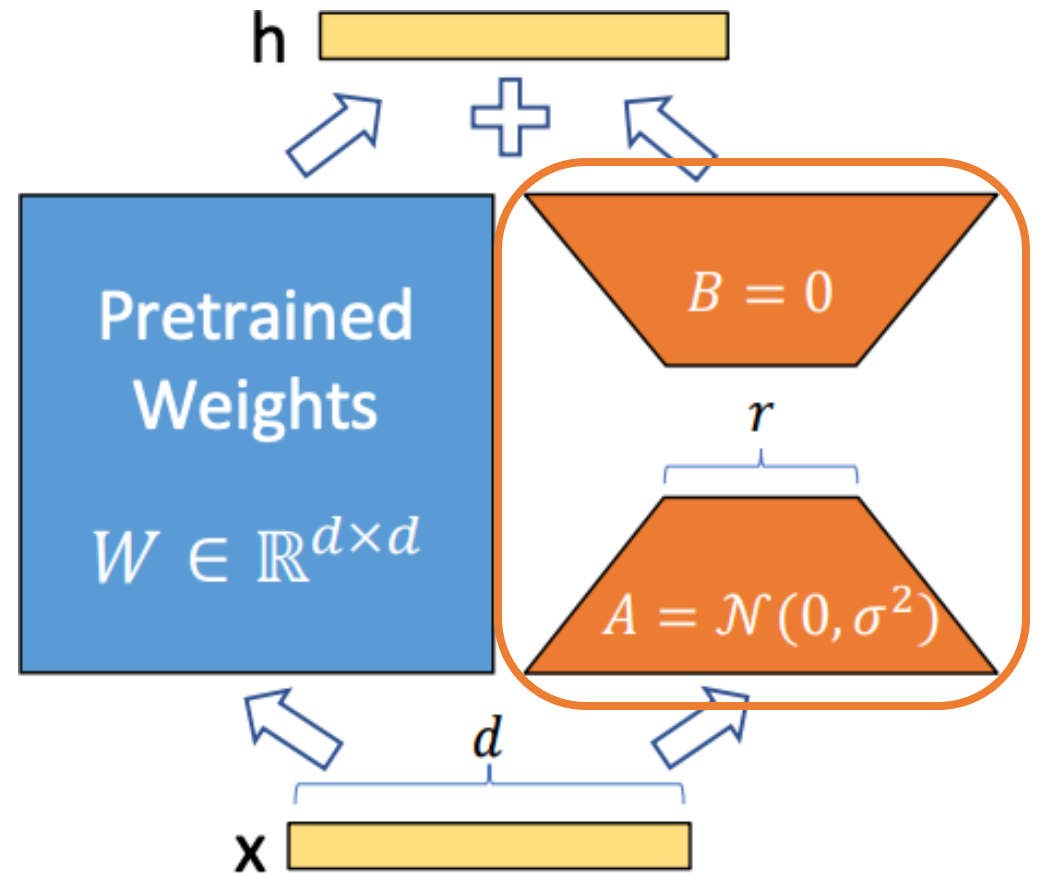
- An MLP layer is the same as multiplying a matrix and applying an activation function.
- **Approximate** the matrix as a multiplication of **two low-rank matrices**.



Low-Rank Adaptation

Low-Rank Approximation:

Equivalent to having two MLP layers with a **low-dimensional intermediate output**.

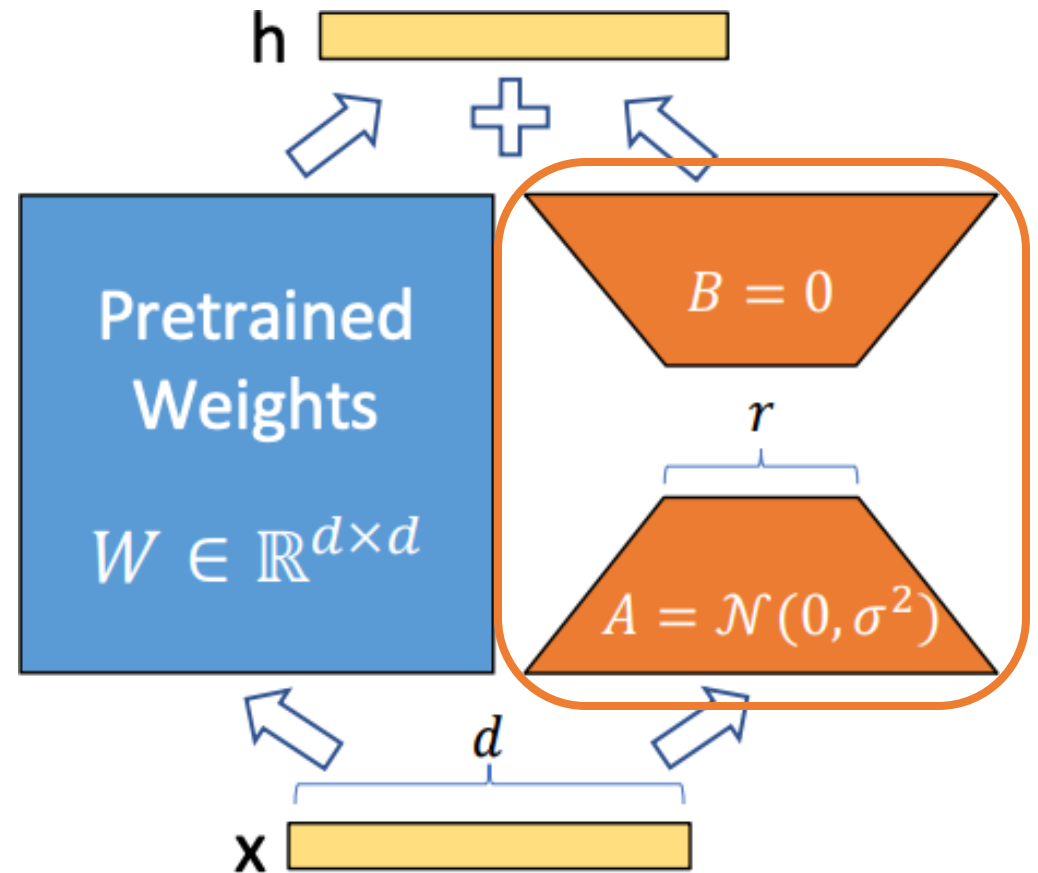


- <https://civitai.com/>

Low-Rank Adaptation

Low-Rank Approximation:

Equivalent to having two MLP layers with a **low-dimensional intermediate output**.



Classifier Guidance

Train a classifier to compute $\nabla_{\mathbf{x}_t} \log p_\phi(y|\mathbf{x}_t)$.

$$\bar{\boldsymbol{\varepsilon}}_\theta(\mathbf{x}_t, t) = \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) - \omega \sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}_t} \log p_\phi(y|\mathbf{x}_t)$$

Train 一個 classifier 來給每一步 denoise 加上一個修正量

優點：不用 paired training data，甚至可以直接加在 unconditional diffusion 上

缺點：需要事先定義好 condition (class)，且最終生成品質也取決於這個 classifier 的好壞

Classifier-Free Guidance (CFG)

Train the noise predictor while taking the additional input y or the null label ϕ as input:

$$\tilde{\epsilon}_{\theta}(x_t, y, t) = (1 + \omega)\hat{\epsilon}_{\theta}(x_t, y, t) - \omega\hat{\epsilon}_{\theta}(x_t, \phi, t)$$

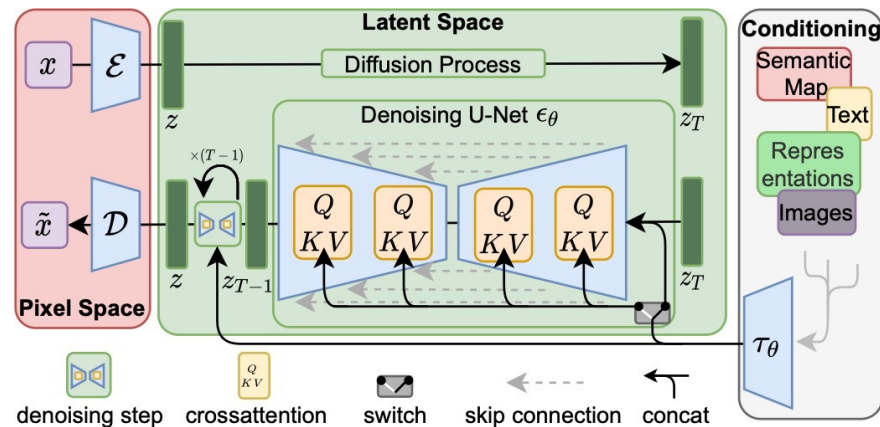
直接讓 noise prediction network 吃 condition

優點：condition 的形式很彈性（text, audio, image, etc.），可以做 condition 外插來強化

缺點：需要有 paired training data，且 train diffusion models 時就要有

Latent Diffusion

- Image encoder/decoder are **pretrained**.
- The **noise predictor** is trained in the **latent space**.
- The input **condition** is incorporated into the **cross-attention** module.



Latent Diffusion

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V$$

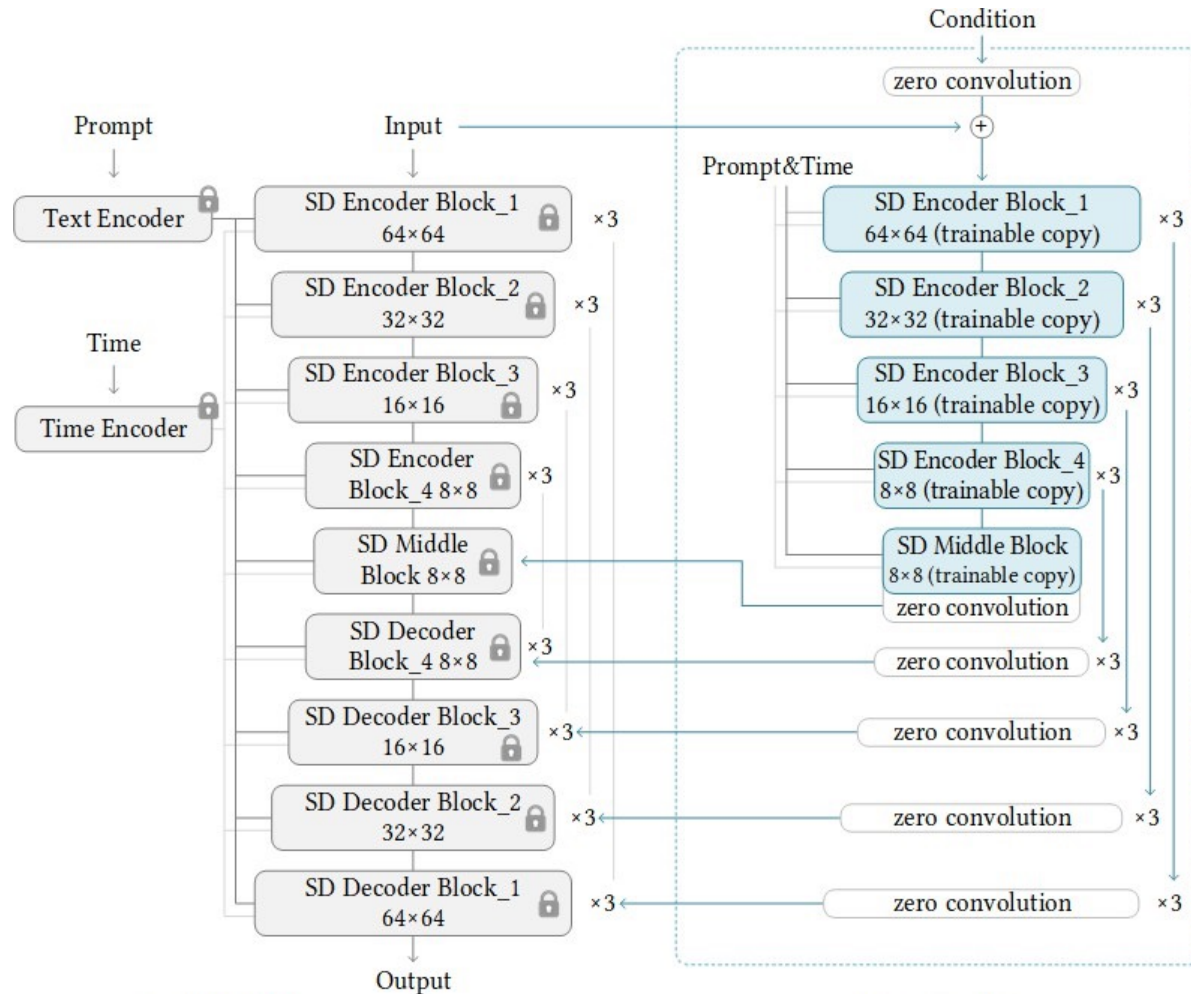
- Query Q : The output of each U-Net layer.
- Key K and Value V : The outputs from the input condition encoder.

ControlNet

Can we **convert** a pretrained unconditional image diffusion model into an **image-conditioned generative model** using a relatively much **smaller** set of input-output pairs ($\sim 100k$)?



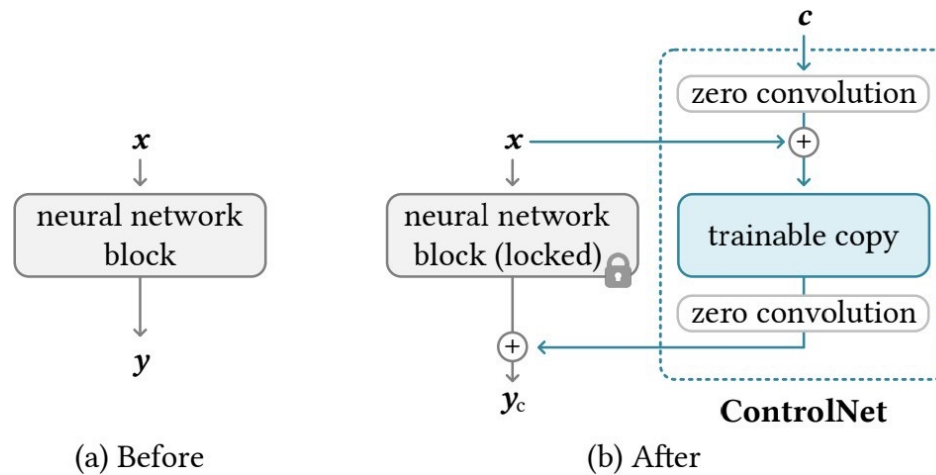
ControlNet



Fully leverage the pretrained noise prediction network to process the **conditional** image.

Zero Convolution

$$\mathbf{y}_c = F(\mathbf{x}; \Theta) + Z(F(\mathbf{x} + Z(\mathbf{c}; a_1, b_1); \Theta_c); a_2, b_2)$$



Low-Rank Adaptation

How to generate images of myself or my objects?

Input images



A [V] backpack in the Grand Canyon



A [V] backpack with the night sky



A [V] backpack in the city of Versailles



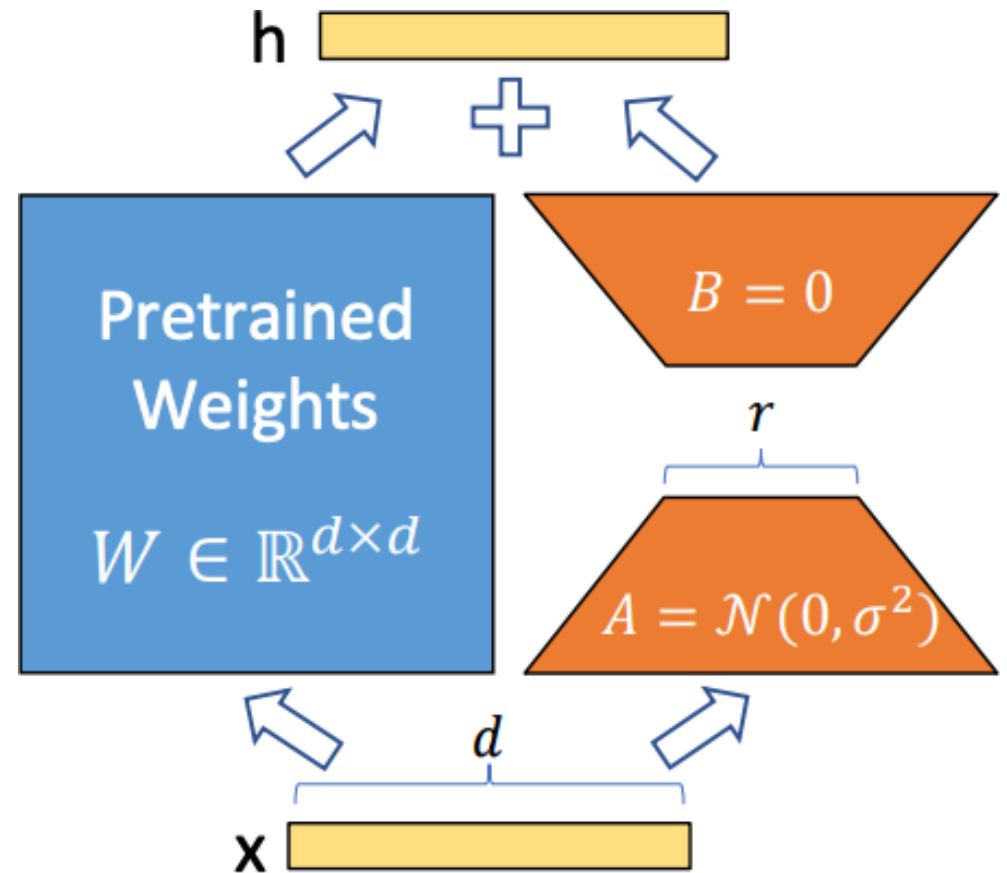
A wet [V] backpack in water



A [V] backpack in Boston

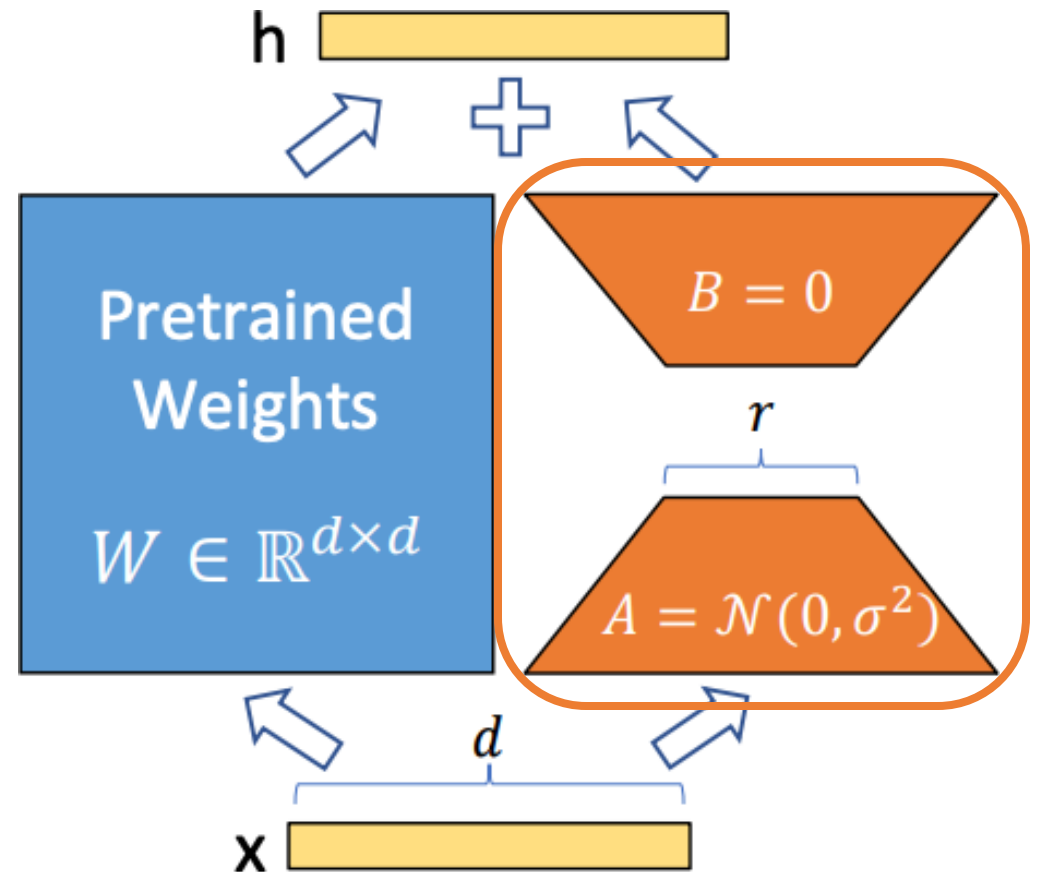
Low-Rank Adaptation

Introduces an **additional branch** in a neural network that takes the same input as the main model.



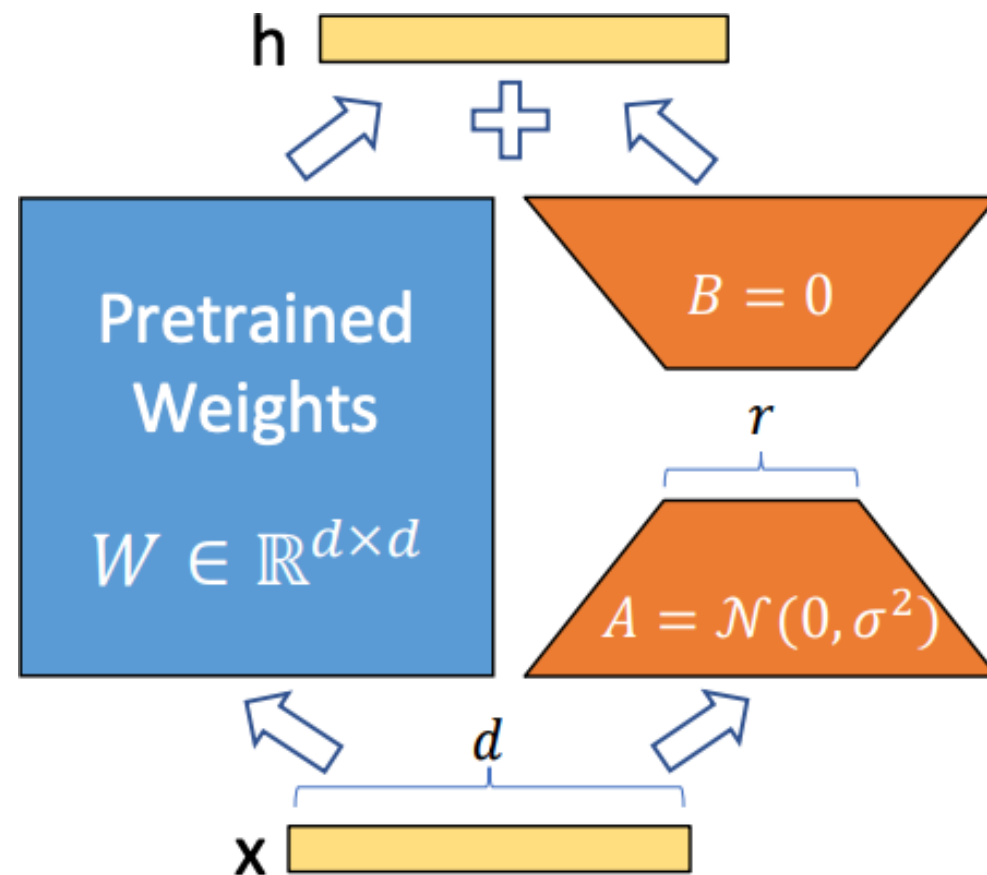
Low-Rank Adaptation

Uses a **bottleneck** architecture where the intermediate outputs are **reduced in dimensionality**.



Low-Rank Adaptation

Q. 為什麼這邊不需要 ControlNet 的 zero convolution 來避免 pre-trained distribution 被破壞？



Low-Rank Adaptation

A. $B = 0$ 確保了初始不影響，與 zero convolution 有同樣的效果

